第四部分

推荐系统工程实践

第 15 章 特征工程

特征工程是推荐系统连接业务数据与模型的关键环节。对于一个工业级推荐系统而言，模型最终能够学习到什么，很大程度上取决于输入特征中包含了什么信息。用户过去看过什么、喜欢什么，物品具有什么属性，当前处于什么场景，以及用户与候选物品之间存在怎样的历史关系，这些信息都需要通过合理的特征形式提供给模型。因此，特征工程并不仅仅是对原始数据进行清洗、编码和转换，更重要的是将复杂的业务信息转化为模型能够有效利用的表示。

在传统的推荐系统中，特征工程具有更加突出的地位。由于早期模型的表达能力有限，算法工程师往往需要手动构造大量离散特征、统计特征和交叉特征，例如用户年龄与物品类目的组合、用户地域与品牌的组合，以及用户在过去一段时间内的点击次数、有效播放次数、购买次数等统计类的信息。模型本身主要负责在人工设计好的特征空间中学习参数，因此“如何设计特征”往往直接决定了模型效果。

但是，随着深度学习的发展，这一模式逐步发生了深刻的变化。由于 Embedding 可以将高维稀疏 ID 映射到低维连续空间，通过 DNN、FM、DCN、Attention 等结构能够自动学习到不同特征之间的复杂交互关系，而 RNN 和 Transformer 可以直接从用户行为序列中学习用户的兴趣表示。此外，多模态模型则可以进一步将文本、图像、音频等非结构化信息转化为 Dense 向量表征。在这种情况下，越来越多过去需要人工设计的特征交叉，就可以交由模型结构来自动学习。

这并不意味着特征工程已经失去意义。恰恰相反，推荐模型的输入正在从简单的 Sparse ID 逐渐扩展到统计特征、行为序列、用户画像、多模态 Embedding 以及实时行为等更加复杂的信息形态。算法工程师需要解决的问题，也从“如何手工构造更多特征”逐渐转变为“哪些信息应该进入模型、应该以什么形式进入模型、如何组织不同时间尺度的行为、如何控制特征成本，以及如何保证训练和线上服务的一致性”。

因此，现代推荐系统中的特征工程正在从传统的**特征工程（Feature Engineering）**逐渐向**特征表示（Feature Representation）**和**特征体系（Feature System）**演进。算法工程师一方面需要设计合理的特征体系，将用户、物品、上下文和行为等信息进行结构化组织；另一方面也需要根据不同信息的特点来选择 Sparse、Dense、Sequence 或 Embedding 等不同的特征表示形式，并通过神经网络完成进一步的特征交互与融合。

从工业实践来看，一个推荐系统中的特征通常可以从两个维度进行理解。第一个维度是**信息来源**，包括用户特征、物品特征、上下文特征、行为特征以及多模态特征；第二个维度是**表示形式**，包括 Sparse 特征、Dense 特征、Sequence 特征以及 Embedding 特征。前者回答“特征描述了什么”，后者回答“这些信息以什么形式提供给模型”。这两个维度共同构成了现代推荐系统的特征体系。此外，特征工程还具有明显的工程属性。一个离线实验中效果良好的特征，如果线上无法稳定计算，或者计算成本过高、更新延迟过大、训练与线上数据分布不一致，那么它仍然很难真正应用于工业系统。因此，特征的生成、存储、更新、服务、监控和治理同样是特征工程的重要组成部分。

本章将围绕现代工业推荐系统中的特征工程展开介绍。首先从特征体系出发，介绍用户、物品、上下文、行为和多模态等不同来源的特征，以及 Sparse、Dense、Sequence 和 Embedding 等不同表示方式；随后讨论特征交互与融合的相关方法，分析从人工特征交叉逐渐向模型自动交互演进的过程；然后会进一步介绍行为统计、用户画像和实时特征等工业实践，并讨论特征筛选、特征成本、数据质量以及训练与线上一致性等问题。需要特别强调的是，本章并不试图罗列所有传统意义上的特征工程方法，而是更加关注在深度学习推荐模型成为主流之后，特征工程在工业推荐系统中的新定位。

15.1 特征体系

推荐系统中的特征本质上是对**用户、物品、场景以及用户行为**等信息的结构化表达。不同业务场景虽然使用的特征类型存在差异，但从整体上看，大多数推荐系统都可以建立由用户特征、物品特征、上下文特征、行为特征和多模态特征构成的基础特征体系。其中，用户和物品描述推荐系统中的核心实体，上下文描述推荐发生

时的环境，行为特征描述用户与系统之间已经发生的交互，而多模态特征则进一步提供文本、图像、音频和视频等非结构化信息。

需要注意的是，传统的“用户特征——物品特征——上下文特征”分类主要回答的是**特征来自哪里**，而现代推荐系统还需要关注**特征以什么形式表示**。同一个业务信息既可以表示为 Sparse ID，也可以表示为 Dense 统计值、Embedding 或者行为序列等等。例如，“用户喜欢篮球”既可以通过篮球 Tag 的 Sparse ID 来表达，也可以通过用户历史篮球内容的统计特征来表达，还可以通过用户行为序列经过 Transformer 编码后得到一个 Dense 兴趣向量。因此，现代特征体系不能仅仅按照业务实体进行划分，还需要结合特征表示形式来进行理解。

15.1.1 用户特征

用户侧特征用于刻画用户的基本属性、兴趣偏好、行为模式以及当前状态，是推荐系统实现个性化的基础。根据用户信息的稳定程度和时间特性，通常可以将用户特征划分为静态用户特征、动态用户特征以及用户表示特征。其中，**用户静态特征**主要描述相对稳定、变化缓慢的用户属性，包括人口统计学信息、账户属性以及设备属性等。例如，年龄、性别、地域、职业、注册时间、会员等级以及设备型号、设备价格区间范围等都属于典型的用户侧的静态特征。这类特征通常具有较强的稳定性，在模型中可以通过 Sparse ID、One-Hot 或者 Embedding 等多种形式来进行表示。

不过，在实际工业推荐系统中，静态用户画像的作用通常没有传统推荐系统中想象得那么绝对。一方面，部分用户属性可能存在缺失或者更新不及时的问题；另一方面，用户的即时兴趣往往比长期稳定属性更加能够决定当前一次推荐行为。例如，同一个用户长期关注美食内容，但在当前的 Session 中该用户连续观看了多个篮球视频，那么此时用户的短期兴趣可能比“用户地域”或者“用户年龄”等静态属性更加重要。因此，现代推荐系统通常需要同时维护用户的长期兴趣属性以及用户短期动态行为的变化信息。

用户动态特征主要反映了用户近期行为和实时状态，是当前工业推荐系统中非常重要的一类特征。典型的动态特征包括用户最近点击、观看、收藏、关注、评论、分享和购买等行为，以及这些行为在不同时间窗口上的统计结果。例如，可以分别统计用户最近 1 天、7 天和 30 天的点击次数、观看次数、平均观看时长以及不同类别内容的消费比例：

$$\text{click}_{1d}, \quad \text{click}_{7d}, \quad \text{click}_{30d} \tag{15.1}$$

$$\text{watch_time}_{1d}, \quad \text{watch_time}_{7d}, \quad \text{watch_time}_{30d} \tag{15.2}$$

不同时间窗口能够反映不同尺度的用户兴趣。较长时间窗口更适合描述用户的长期偏好，而较短时间窗口则更加敏感，可以反映用户近期兴趣变化。因此，在工业推荐系统中，经常需要同时构造多个时间尺度的行为特征。这种基于时间窗口的多尺度统计特征构建方法在实际工业界推荐系统中非常常见，在推荐系统 base 不是很高的情况下，引入这些特征也能够显著提升模型的表达能力以及线上 A/B 效果。

除了统计特征之外，用户行为序列也是现代推荐系统中非常重要的输入形式。给定用户最近的行为序列，可以表示为：

$$S_u = \langle item_1, item_2, \ldots, item_n \rangle \tag{15.3}$$

其中每个 $item_i$ 不仅可以包含物品 ID，还可以进一步携带物品类别、Tag、聚类 ID、作者、行为类型、观看时长以及时间间隔等信息。与将用户历史行为压缩成几个统计值相比，行为序列能够保留更加完整的行为顺序和上下文信息，因此已经成为现代序列推荐和推荐模型的重要输入。基于用户行为序列的建模方法将会在后续第20章中进行详细介绍。

进一步地，用户特征还可以被编码为更加紧凑的**用户 Embedding**。例如，可以通过用户历史行为序列、兴趣标签或者多兴趣模型学习用户的 Dense 表征。与传统用户画像相比，Embedding 不再要求人工定义“用户喜欢什么”，而是通过模型直接从大量历史行为中学习潜在兴趣空间。这也是传统用户画像逐渐向表示学习演进的重要体现。

15.1.2 物品特征

物品特征用于描述推荐候选内容本身，是模型判断“这个物品是否适合当前用户”的重要依据。在不同推荐场景中，物品可以是短视频、直播间、商品、文章、音乐或者广告，因此具体特征会存在较大差异，但总体可以划分为基础属性、关系特征、内容表示、质量与统计特征等几类。

基础属性特征主要描述物品的结构化元信息。例如，短视频可以包含 Photo ID、作者 ID、类目、Tag 标签、内容垂类 Cluster、发布时间、视频时长等信息；电商商品可以包含商品 ID、品牌、类目、价格、店铺、历史单量等信息。这类特征通常具有较强的业务语义，是推荐模型中最基础的一类 Sparse 特征。

物品关系特征用于描述物品与其他物品之间的关联关系。例如，两个视频是否属于相同的垂类 Cluster、是否具有相同 Tag、是否在用户历史行为中具有共现关系，或者两个商品是否具有协同购买的关系等等。这些关系既可以人工构造成离散交叉特征，也可以通过协同过滤、图模型或者 Embedding 模型进行表示。

内容表示特征则进一步将物品自身的内容转化为 Dense 表征。例如，可以利用文本模型对标题、描述和字幕进行编码，利用视觉模型对图片和视频帧进行编码，利用音频模型对直播或视频的音频进行提取和编码等。然后再将不同模态的信息进行融合，形成统一的物品侧 Embedding。随着多模态预训练模型以及多模态大模型的发展，这类 Dense 表征已经逐渐成为现代推荐系统中重要的物品特征来源，同时也是推荐系统与大模型结合的重要接口。

质量与统计特征则用于描述物品自身的质量和历史表现。例如短视频的播放量、完播率、平均观看时长、点赞率、评论率、分享率等，商品的点击率、转化率、销量和评分等，都可以作为物品侧的统计特征。在短视频推荐中，关于视频的完播率、点赞率、评论率这些信息，我们通常也称之为“empirical xtr”特征，即基于历史经验统计得到的特征。这类“empirical xtr”特征通常也可以看做是后验 xtr，而粗精排主模型预估出来的是先验 pxtr。这类关于物品的质量与统计特征通常具有明确的数值含义，因此相比简单地将其离散化为 Sparse ID 特征来说，这类特征更适合采用 Dense 特征进行表示，或者基于数据分布情况进行分桶然后将同一个桶内的取值都映射相同的 Embedding 向量上。

还需要特别注意的是，这些统计特征通常具有明显的时间属性。例如，一个视频在发布后第 1 小时和发布后第 30 天的播放量具有完全不同的含义。因此在工业级推荐系统中通常还会结合时间窗口、时间衰减以及平滑策略构造更加稳定的统计特征，同时需要注意新物品由于历史数据不足产生的冷启动问题。当一个业务刚开始起步的时候，推荐系统中积累的用户历史行为数据通常处于偏少的状态，这时统计特征的可靠性也会相应下降，无论是基于时间窗口的用户交互统计特征还是仅物品侧基于时间窗口聚合的统计特征，都会面临数据稀疏的问题。如果这些统计特征大部分取值都为 0 或者较小的数值，那么模型在训练中通常也很难从这种极其长尾的统计特征中捕捉到有效信息，模型中引入这些特征通常在线上的 A/B 测试中也难以体现其价值。

15.1.3 上下文特征

上下文特征通常描述的是推荐发生时的环境信息，用于回答“用户是在什么场景下看到这个物品”的问题。即使用户和物品本身没有发生变化，不同的时间、设备、页面位置、Session 状态以及推荐候选集合状态也可能导致完全不同的用户行为。

时间特征是最常见的上下文特征之一，包括小时、星期、工作日/周末、节假日以及距离上一次行为的时间间隔等。例如，同一个用户在工作日上午和深夜的内容消费偏好可能存在明显差异。除了绝对时间，还可以通过时间间隔描述行为的新鲜程度，例如：

$$\Delta t = t_{\text{now}} - t_{\text{last_action}} \tag{15.4}$$

设备与网络特征描述用户当前使用的技术环境，包括设备型号、操作系统、App 版本、网络类型、网络质量以及屏幕尺寸等。在短视频、直播等强实时业务中，设备和网络环境还可能影响视频加载、观看时长以及交互行为，因此这类上下文信息不仅影响用户偏好，也可能影响用户行为本身。这类设备与网络特征在工业界推荐系统应用时，不仅会在粗精排模型中使用，在客户端的端排序模型中也通常会使用到这些信息，根据用户的设备与网络状态进行自适应的排序。

场景与 Session 特征则更加关注用户当前所处的推荐场景。例如当前页面、推荐入口、Session 持续时间、当前 Session 已经观看的内容数量，以及最近连续行为等，都可以用于刻画用户即时意图。相比长期用户画像，这类特征具有更强的时效性，是现代推荐系统实现短期兴趣建模的重要组成部分。

推荐候选集合状态特征关注的是当前推荐候选集合的整体属性和分布情况。例如，候选集合中物品的数量、类别分布、热门程度、平均质量评分、empirical xtr 分布等。这类特征有助于模型理解当前推荐环境，从而更好地判断某个物品在特定候选集合中的相对排序。在粗精排模型的 LTR 模型中，通常会将这些特征作为全局上下文信息输入，从而辅助模型进行更准确的排序决策。同时也会让模型不仅感知到每个物品各自 Pointwise 的信息，同时也能够进行这种 Context-Aware 建模，使得每个参与排序的物品都能够感知候选集合的整体状态。这样引入了上下文增量信息之后，LTR 模型通常能够在排序精度上获得显著提升。

除此之外，推荐候选集状态特征还是一种重要的上下文信息，它能够帮助模型理解当前推荐环境的整体情况，从而在排序和推荐决策中发挥更大的作用。比如，在链路一致性建模中，粗排的 LTR 模型可以利用精排阶段候选集的状态特征来调整自身的排序策略，从而实现更一致的排序结果，在线上 A/B 实验中也能够取得一定的指标提升。

15.1.4 行为特征

与传统推荐系统相比，现代工业级推荐系统越来越重视用户的**行为特征**。原因在于用户真实发生过的行为通常比静态画像更加能够反映用户当前的兴趣。尤其是在短视频、直播和信息流推荐中，用户的兴趣变化非常快，仅依赖年龄、性别、地域等静态属性很难准确描述用户当前的消费意图。因此，充分使用用户的行为特征可以极大提升推荐系统中模型预估的精度。

通常来说，用户行为特征可以按照行为类型、时间窗口和行为强度进行进一步划分。按照行为类型，可以包括曝光、点击、观看、点赞、评论、收藏、关注、分享、购买等。不同行为具有不同的反馈强度，例如“曝光”通常只能说明用户看到了内容，而“点赞”、“关注”甚至“购买”能够提供更加明确的正反馈。因此，可以针对不同的行为类型分别进行统计或者建模。

按照时间窗口，我们可以构造：

$$\text{Feature}_{1d}, \quad \text{Feature}_{7d}, \quad \text{Feature}_{30d} \tag{15.5}$$

分别描述用户近期、中期和长期行为。进一步地，还可以结合时间衰减函数，使距离当前时间越近的行为获得更高权重：

$$w(\Delta t) = \exp(-\lambda \Delta t) \tag{15.6}$$

从而降低过期行为对当前推荐决策的影响。在具体实践的时候，如果推荐系统中的用户行为数据量级非常大，那么也可以使用基于相对时长的位置编码（Position Encoding）来进一步表示行为的时间信息，从而在模型中更好地捕捉行为的时序特性。但如果推荐系统中的用户行为数据量级相对较小，则使用位置编码的方式可能还不如直接使用上述的事件衰减函数这种人工设计的函数来引入时序信息。

按照行为强度，我们也可以将用户行为特征进一步区分为简单交互和深度交互。例如，对于短视频而言，仅仅点击一个视频与观看 30 秒、观看 90 秒甚至完整观看一个视频所提供的信息量并不相同。因此，观看时长、观看比例、连续观看次数、消费深度等都可以作为重要的行为特征。这也是当前推荐系统中从 Sparse 特征向 Dense 行为特征演进的一个典型方向。对于“用户过去 7 天点击了多少次体育内容”这类具有明确数值语义的信息，如果将其简单编码成一个离散 ID，不仅会损失原始数值关系，还会引入额外的 Embedding 参数。相比之下，直接将统计结果作为 Dense 行为特征输入模型，通常更加自然，也更加符合该信息本身的语义。

在实际工业级推荐系统优化中，我们还可以进一步构造用户历史行为与候选物品属性之间的 Dense 交叉特征。例如，统计用户近期观看某个 Cluster、Tag 或者类目内容的次数、观看时长以及占比，再与当前候选物品的 Cluster、Tag 和类目进行匹配，从而得到：

$$f(u, i) = \text{Agg}\left(\mathcal{H}_u \cap \mathcal{A}_i\right) \tag{15.7}$$

其中 $\mathcal{H}_u$ 表示用户历史行为集合，$\mathcal{A}_i$ 表示候选物品的属性集合，$\text{Agg}(\cdot)$ 表示对匹配行为进行统计聚合。这类特征

本质上是在显式表达“用户过去是否对当前物品所属内容感兴趣”，在工业推荐系统中通常具有较强的业务解释性和实际价值。当然，我们也可以使用用户行为历史的物品 ID、作者 ID 序列与当前候选的物品 ID、作者 ID 特征进行交叉，从而得到更加细粒度的兴趣匹配特征。这种方式唯一存在的问题就是，基于用户历史序列的 ID 特征与候选 ID 特征交叉之后，会导致该特征可能的取值非常多，从而带来稀疏性、计算复杂度、以及 Sparse Embedding 存储压力等方面的挑战。

15.1.5 多模态特征

随着短视频、电商和直播等内容型推荐业务的发展，推荐系统所处理的信息已经从结构化 ID 扩展到文本、图像、音频和视频等多模态的数据。传统的视频 ID、Tag ID 或者类别 ID 只能描述内容的有限结构化属性，而多模态特征能够直接利用内容本身的信息。文本特征可以来自标题、描述、字幕、OCR 文本以及用户评论等；视觉特征可以来自图片、视频帧以及场景识别结果；音频特征则可以来自语音、背景音乐和声音事件等。通过专门的预训练模型、内容理解模型或者视觉语言模型（Vision Language Model，VLM），可以将这些信息编码为 Dense Embedding：

$$e_i^{text} = f_{text}(x_i^{text}) \tag{15.8}$$

$$e_i^{vision} = f_{vision}(x_i^{vision}) \tag{15.9}$$

$$e_i^{audio} = f_{audio}(x_i^{audio}) \tag{15.10}$$

随后可以通过拼接、加权、Attention 或者更加复杂的多模态融合网络得到统一的物品表示：

$$e_i = f(e_i^{text}, e_i^{vision}, e_i^{audio}) \tag{15.11}$$

多模态特征的价值不仅在于提升模型对内容的理解能力，也能够缓解传统 ID 特征在冷启动场景下的问题。对于一个刚刚发布、尚未积累足够用户行为的新视频，仅依赖视频 ID 或者历史统计特征很难判断其内容质量和用户兴趣，而文本、图像和视频本身的语义信息可以提供额外的泛化能力。

尤其是在最新的生成式推荐以及 LLM4Rec 等研究中，多模态特征的作用愈发凸显。通过将文本、图像、音频和视频等多模态信息输入大模型，推荐系统能够更好地理解内容语义，从而在冷启动和长尾场景下提供更强的泛化能力。在工业界推荐系统最新的实践中，算法工程师也在尝试将多模态表征与传统推荐、生成式推荐模型相结合，从而进一步提升物品的识别与挖掘能力。这里需要强调的是，通常来讲在工业界推荐系统中大规模引入多模态表征会存在一定的计算与存储成本，因此如果从工业界落地 ROI 的视角来看，需要权衡多模态特征带来的收益与其额外的资源消耗。在一些比较成熟的推荐系统框架中，现有的 base 非常高，仅通过引入多模态表征进行增量信息挖掘的能力通常有限，因此需要结合大的链路架构改造才可以拿到线上 A/B 测试的显著效果。而在推荐系统 base 较低的情况下，多模态特征的引入一般会带来更加明显的推荐效果增益。

15.2 特征表示

在确定推荐系统需要使用哪些特征之后，还需要进一步解决一个问题：

笔记 这些特征应该以什么形式输入模型？

同样的业务信息，采用不同的表示方式可能会产生完全不同的模型效果和工程成本。例如，用户过去 7 天观看了多少个体育类视频，可以直接作为一个 Dense 统计值输入模型，也可以离散化后作为 Sparse 特征处理；用户过去观看过哪些视频，则可以表示成一组 Sparse ID 特征，也可以组织成行为序列特征、再通过 Transformer 进一步学习用户兴趣 Embedding。因此，从这些例子我们可以看出，特征表示并不是简单的数据格式转换，而是连接原始业务信息与模型表征学习的重要环节。

从模型输入的形式来看，推荐系统中的特征通常可以划分为 Sparse 特征、Dense 特征、Sequence 特征和 Embedding 特征。其中，Sparse 特征主要用于表示 ID、类别、Tag 等离散的信息；Dense 特征主要用于表示具有明

确数值语义的连续信息；Sequence 特征用于保留用户历史行为的顺序和上下文等信息；Embedding 特征则进一步将离散实体、行为序列或者非结构化的内容映射到低维连续向量空间。需要注意的是，这几类特征并不是严格互斥的。例如，一个用户 ID 最初属于 Sparse 特征，经过 Embedding 层之后就会变成 Dense 向量特征；一个用户行为序列经过 Transformer 编码后，也可以进一步得到用户 Embedding。因此，更准确地说，这几类概念分别描述了特征的原始形式、组织方式以及模型学习后的表示形式。

从推荐系统的发展过程来看，特征表示也经历了从**人工设计的 Sparse 特征**向**模型学习的 Dense 表示**逐渐演进的过程。早期的工业级推荐模型高度依赖人工设计的离散特征与交叉特征，模型主要负责在固定的特征空间中学习参数。随着 Embedding 表示、深度神经网络、序列建模以及多模态模型的快速发展，越来越多的信息可以直接通过神经网络学习得到连续表示，特征工程的重点也逐渐从“如何构造更多特征”转向“如何提供更加有效的信息以及如何选择合适的表示方式”。而在具体的特征构造形式的探索以及交叉特征的手工构造方面的工作越来越少，更多的研究和工程实践开始关注如何设计高效的特征表示学习和特征交互学习方法，以及如何将多模态信息与用户行为序列进行有效结合等方向。

15.2.1 特征表示形式

除了前一章节中按照用户、物品和上下文等信息来源进行分类之外，从模型输入的形式来看，工业级推荐系统中的特征主要可以分为 Sparse 特征、Dense 特征、Sequence 特征和 Embedding 特征。不同表示方式并不存在绝对的优劣，而是应该根据特征本身的语义、数据规模、稀疏程度以及下游模型结构进行选择。

Sparse 特征主要用于表示 ID、类别、Tag 等离散信息。例如用户 ID、设备 ID（DID）、视频 ID、作者 ID、类别 ID 以及 Tag ID 等通常具有非常大的取值空间，但对于单个样本而言，实际激活的特征数量非常有限，因此适合采用 Sparse 表征。以用户 ID 为例，可以首先将用户映射为离散索引 x_u，再通过 Embedding 矩阵 E 将其映射为低维连续向量：

$$e_u = E[x_u] \tag{15.12}$$

其中 $E \in \mathbb{R}^{|\mathcal{U}| \times d}$ 表示用户 Embedding 矩阵，$|\mathcal{U}|$ 表示用户词表大小，d 表示 Embedding 维度。类似地，Item ID、Author ID、Category ID 等都可以采用相同的方式进行 Embedding 化。Sparse 特征的核心优势在于其较强的**记忆能力（Memorization）**。对于具有大量历史交互数据的用户，User ID Embedding 能够直接学习该用户长期积累下来的个性化偏好；对于热门物品，Item ID Embedding 也能够通过大量历史曝光、点击、观看和转化行为学习该物品的整体表现。因此，在工业推荐系统中，ID 类 Sparse 特征往往具有非常重要的作用。

但 Sparse 特征也存在明显的问题。首先，其参数规模通常与 ID 词表大小直接相关。例如，如果用户规模达到 10^8，即使每个 User Embedding 只有 64 维，也会产生非常庞大的 Embedding 参数量：

$$N_{\text{param}} = |\mathcal{U}| \times d \tag{15.13}$$

因此，工业级推荐系统通常需要通过 Embedding 压缩、参数共享、低维 Embedding、频次截断、哈希以及其他 Embedding 优化技术来控制存储和计算成本。其次，Sparse 特征通常存在明显的长尾和冷启动问题。对于交互次数很少的新用户、新物品或者长尾 ID，模型很难仅依赖 ID 本身学习到稳定的 Embedding 表示。此时，类别、内容、行为统计和多模态 Embedding 等更加具有泛化能力的特征就显得尤为重要。

Dense 特征主要用于表示具有明确数值语义的信息，例如观看次数、点击次数、平均观看时长、CTR、CVR、内容质量分数、价格以及距离上一次行为的时间间隔等。这类特征本身具有连续或者有序的数值关系，因此通常不需要像 ID 一样为每一个取值建立独立的 Embedding，而是可以直接输入模型，必要时进行归一化、截断、Log 变换或者其他数值变换。例如，用户最近 7 天的观看次数可以直接表示为：

$$f_u^{watch} = N_u^{watch}(7d) \tag{15.14}$$

用户最近一段时间观看某类内容的平均观看时长则可以表示为：

$$f_u^{duration} = avg(duration_i) = \frac{1}{N_u} \sum_{i=1}^{N_u} duration_i \tag{15.15}$$

与将这些统计值直接离散化成 Sparse ID 相比，Dense 特征能够更加直接地保留原始数值之间的关系。例如，观看次数为 100 和 101 通常应该具有相近的语义，而不是像两个完全独立的类别一样进行建模。因此，对于具有明确数值含义的统计特征，采用 Dense 表示通常会更加自然一些。但是，Dense 特征也并不意味着维度越高越好。在工业推荐系统中，大量 Dense 特征通常是通过数据分析、行为统计和业务规则挖掘所得到的，单个 Dense 特征往往只有一个或者几个维度，其总维度通常来说会远远低于由大量 ID Embedding 所产生的 Dense 向量。因此，Dense 特征的价值更多体现在**信息密度和业务语义**上，而不是单纯依赖维度规模。

Sequence 特征用于表示用户历史行为序列，是现代推荐系统区别于传统静态特征体系的重要形式。给定用户最近的行为，可以表示为：

$$S_u = \langle item_1, item_2, \ldots, item_n \rangle \tag{15.16}$$

其中 $item_i$ 表示用户第 i 次历史交互的物品。除了 Item ID 之外，每个行为还可以进一步携带 Category、Tag、Author、Behavior Type、Request Type、Watch Duration 以及 Timestamp 等信息。

序列特征相比起简单的统计特征会具有更加丰富的信息表达能力。统计特征通常会将大量历史行为压缩为少量的数值，例如“最近 7 天观看体育视频 20 次”，但这种表示方式一方面会有信息损失的问题，另一方面也无法区分用户具体观看了哪些内容，也无法区别出这些行为发生的先后顺序。而序列特征可以保留完整的行为轨迹，使模型能够进一步学习用户的短期兴趣、长期兴趣以及兴趣演化过程。例如，对于短视频推荐而言，可以将用户序列表示为如下形式：

$$S_u = \langle photo_1^{sports}, photo_2^{sports}, photo_3^{game}, photo_4^{sports} \rangle \tag{15.17}$$

对粗精排模型来说，模型不仅可以知道用户观看过体育与游戏的内容，也可以进一步学习到用户最近存在从游戏兴趣转向体育兴趣的兴趣迁移行为。这些动态兴趣通常很难通过传统的静态画像或者简单统计特征完整表达。在具体建模时，Sequence 特征通常需要结合专门的序列建模网络进行处理，例如 RNN、Attention、Transformer 以及各种兴趣提取网络。经过序列模型编码后，可以得到用户当前行为序列对应的 Dense 表征：

$$e_u^{seq} = f_\theta(S_u) \tag{15.18}$$

其中 f_θ 表示序列编码模型。此时，原始的序列特征进一步转化为了用户 Embedding 或者用户兴趣 Embedding。

Embedding 特征则是推荐系统中最为常见的 Dense 表征之一。Embedding 的核心思想如下：

定义 15.1

Embedding 表征是将高维、离散或者复杂的信息映射到低维连续向量空间，使得模型可以在连续表示空间中学习不同实体之间的潜在关系。

在工业级推荐系统中，Embedding 的来源是非常丰富的。最简单的是通过 ID Embedding 将用户 ID、物品 ID、作者 ID 和类别 ID 映射为 Dense 向量；进一步可以通过序列模型得到用户 Embedding 或者兴趣 Embedding，通过图模型得到图 Embedding，通过文本模型得到文本 Embedding，通过视觉模型得到图片 Embedding，通过多模态模型得到多模态 Embedding。例如，可以分别得到用户、物品和内容的 Embedding：

$$e_u = f_u(x_u), \qquad e_i = f_i(x_i), \qquad e_i^{content} = f_c(x_i^{content}) \tag{15.19}$$

随后将这些表示输入排序网络，通过 MLP、Attention、Cross Network 或者其他交互结构进一步学习用户与物品之间的匹配关系。需要特别强调的是，Embedding 并不只是稀疏特征的一种编码技巧。从更广义的角度来看，Embedding 代表了一种**从原始信息到连续语义空间的表征学习方式**。一个用户 ID 经过 Embedding 层得到的向量、一段用户行为序列经过 Transformer 得到的向量，以及一个视频经过多模态模型得到的向量，本质上都属于 Dense 表征，只是其信息来源和学习方式不同。

因此，从整个推荐系统的发展来看，可以观察到一个比较明显的演进趋势：

$$\text{Sparse ID} \rightarrow \text{Embedding} \rightarrow \text{Dense 表征} \rightarrow \text{序列表征} \rightarrow \text{多模态表征} \tag{15.20}$$

传统推荐系统更加依赖人工设计的稀疏特征和交叉特征，而现代推荐系统则越来越依赖神经网络自动学习稠密表征以及复杂特征交互。特征工程的重点也因此逐渐从“如何人工构造更多特征”转向“如何将有价值的信息以更

加适合模型学习的形式提供给模型”。

15.2.2 特征体系的工业化设计

在工业级推荐系统中，特征体系通常不是简单地将所有能够获取的数据全部输入模型，而是需要综合考虑信息价值、时效性、数据规模、存储成本、计算成本以及后续线上推理的能力。一个典型的工业推荐模型可以同时包含：

用户稀疏特征 + 物品稀疏特征 + 稠密统计特征 + 行为序列特征 + Embedding 特征 + 上下文特征 (15.21)

例如，在短视频推荐场景中，用户侧可以同时包含 UID、DID、用户长期兴趣 Embedding、最近观看序列以及不同时间窗口的观看统计；物品侧可以包含 Photo ID、Author ID、Cluster、Tag、视频时长、历史观看统计以及多模态 Embedding；上下文侧则可以包含时间、设备、网络、Session 状态等信息。

不同类型的特征还可以由不同的网络结构进行处理。例如，Sparse 特征通常通过 Embedding Layer 进行编码，Dense 特征可以经过归一化和 MLP 进行映射，Sequence 特征可以使用 Transformer 或者 Attention 网络来进行建模处理，多模态 Embedding 则可以通过专门的融合模块进行处理，最终再将不同来源的表示进行融合。这种“不同特征使用不同网络处理”的设计在工业推荐系统中越来越常见。不同特征具有不同的数据分布和信息密度，如果简单地将所有特征直接 Concat 后输入一个统一的 MLP，虽然实现简单，但并不一定能够充分利用不同特征的特点。因此，工业模型通常会针对不同特征组设计相对独立的处理模块，再通过 Concat、Attention、Gating 或者其他的融合方式形成统一的用户与物品表示。

在推荐模型没有“大模型化”之前，通常粗精排模型都会有上述各种精巧的小模块结构来进行细粒度的特征表示学习与特征交互建模。例如，对于新增的一批 Dense 行为交叉特征，可以在精排模型中增加一个轻量级的 Dense 特征子网络，对这部分特征进行单独建模；进一步地，也可以采用小规模 MMoE，根据不同的预测目标学习不同的 Dense 表示，再与原有的用户、物品和上下文 Embedding 进行融合。这种设计并不是简单地“增加特征”，而是针对不同特征的数据分布、业务语义和任务相关性选择更加合适的表示与建模方式。

15.2.3 Embedding 表征的作用

在工业级推荐系统中，由于 Embedding 已经成为处理离散 ID 特征的标准方法，因此模型输入中的 Embedding 特征通常占据了较大的维度，并且在特征交互以及最终模型性能中发挥着非常关键的作用。这一现象与工业推荐系统中的数据规模密切相关。在拥有海量用户和物品的推荐场景中，UID、DID、Item ID、Author ID 等 ID 特征通常拥有大量历史行为数据，因此其 Embedding 能够对用户和物品的历史表现进行非常细粒度的记忆。以用户侧为例，UID 和 DID 等 ID 特征的 Embedding 可以帮助模型学习用户长期积累下来的个性化偏好。不同用户即使具有相同的年龄、地域、设备和其他画像属性，其历史消费行为也可能完全不同，而 UID Embedding 能够直接利用用户历史数据学习这种个体之间的差异。因此，在很多工业推荐模型中，如果直接去掉 UID 或者 DID 等重要 ID 特征，模型往往会出现比较明显的效果下降。

从模型学习角度来看，这类 ID Embedding 承担了非常重要的**记忆能力（Memorization）**作用。它能够记忆大量用户和物品在历史数据中的个体化行为模式，而其他 Dense 统计特征、内容 Embedding 或者类别特征则更多承担**泛化能力（Generalization）**作用。二者共同构成现代推荐模型中典型的“记忆 + 泛化”能力，推荐表征进而可以简单地表示为：

$$\text{推荐表征} = \text{记忆能力} + \text{泛化能力} \tag{15.22}$$

其中，ID Embedding 通常具有较强的记忆能力，而 Category、Tag、Content Embedding、多模态 Embedding 以及 Dense 统计特征等则能够帮助模型学习更加具有泛化能力的模式。

因此，在工业实践中不能简单地认为“Dense 特征比 Sparse 特征更加先进”，也不能因为神经网络已经具备较强的表征学习能力就轻易删除传统的 ID Embedding。二者解决的问题并不完全相同。Dense 特征通常能够以更加紧凑的方式表达具有明确业务语义的统计信息，而 ID Embedding 则能够提供非常细粒度的个体记忆能力。

对于物品侧而言，Item ID Embedding 通常也是模型中非常重要的一类特征。一个 Item 不仅具有 Category、Tag、Author 以及内容 Embedding 等通用属性，还可能具有非常特殊的个体表现。例如，同属于“体育”垂类的两个视频，其点击率、观看时长和用户群体可能存在明显差异。Item ID Embedding 能够直接利用历史交互数据记忆这种 Item 级别的差异，因此对于模型学习物品的基础属性以及进行精细的 pxtr 预估通常都非常重要。特别是在精排模型中，模型需要针对当前用户和候选 Item 进行更加精细的个性化判断。此时，Item ID Embedding 可以提供非常直接的 Item-level 信息，而 User ID Embedding 则提供 User-level 信息，二者再通过 DNN、Cross Network、Attention 等结构进行交互，可以形成较强的用户与物品匹配的能力。

当然，ID Embedding 并不是在所有场景下都一定有效，其实际收益与数据规模和 ID 本身的稳定性密切相关。一个典型的例子是直播推荐场景中的 Live ID。与短视频或者商品相比，直播间具有更加明显的实时性。一个 Live ID 对应的内容会随着主播当前的直播状态持续变化，同一个直播间在不同时间点的内容、观众结构和互动状态可能存在较大差异。如果 Live ID 本身的数据积累不足，或者直播内容变化速度远高于 Embedding 更新速度，那么 Live ID Embedding 就可能很难准确学习到稳定的语义表示。在这种数据相对稀疏或者内容快速变化的场景中，直接使用 Live ID Embedding 有时反而可能使模型过度依赖 ID 记忆，导致模型泛化能力下降。此时，去掉 Live ID Embedding，转而更多依赖主播属性、直播内容特征、实时互动统计、用户兴趣以及多模态内容 Embedding，有可能让模型学习到更加稳定的泛化模式。但这并不意味着 Live ID Embedding 本身没有价值。在数据规模较大、直播间历史交互足够丰富，并且 Live ID 具有相对稳定的生命周期和足够更新能力的场景中，Live ID Embedding 仍然可以学习到非常有效的个体化信息，通常也是模型中的重要特征。因此，工业实践中不应该简单地根据“ID 特征容易过拟合”这一原则去删除 ID，而应该结合具体场景的数据规模、ID 生命周期、行为密度以及线上更新机制，通过离线消融实验和线上 A/B 实验判断其实际价值。

从这一角度来看，ID Embedding 的价值本质上取决于**记忆能力与泛化能力之间的平衡**。当一个 ID 拥有大量稳定的历史行为时，Embedding 能够提供非常强的个性化记忆；当一个 ID 数据稀疏、生命周期短或者语义变化快速时，Embedding 则更容易出现过拟合或者表示不稳定的问题。此时，需要通过 Category、Tag、Content、Dense Statistics、Sequence 以及 Multimodal Embedding 等特征提供更加具有泛化能力的信息。因此，在工业推荐系统中，一个更加合理的特征设计原则并不是“Sparse 还是 Dense”，而是根据不同信息的特点组合使用。通过 Sparse ID 保留个体化记忆，通过 Dense Statistics 提供明确的行为语义，通过 Sequence 来保留用户动态兴趣，再通过 Semantic 和 Multimodal Embedding 来增强模型的泛化能力。不同类型的特征在模型中可以相互补充，共同构成了现代推荐系统模型的输入表示。

15.2.4 从特征工程到表征学习

随着深度学习模型能力的不断增强，工业级推荐系统中的特征表示正在发生一个更加深层次的变化：过去工业级推荐系统优化需要人工定义的很多特征，但后续逐渐发展为通过深度模型自动来完成表示学习。传统的特征工程主要关注如何从原始数据中提取有用的特征。例如，工程师需要人工定义用户过去 7 天点击次数、用户对某个垂类的偏好程度，以及用户和候选物品之间的垂类匹配关系等。因此，传统推荐模型的范式主要是：

$$\text{Raw Data} \rightarrow \text{Hand-crafted Feature} \rightarrow \text{Model} \tag{15.23}$$

而现代推荐模型越来越倾向于：

$$\text{Raw Behavior} \rightarrow \text{Representation Learning} \rightarrow \text{Model Prediction} \tag{15.24}$$

例如，直接将用户历史行为序列输入 Transformer，由模型自动学习用户兴趣；将文本、图像和视频输入大规模预训练模型，由大模型自动提取出内容 Embedding；或者将不同特征分组输入 Attention 或者其他交互网络，由模型自动学习哪些特征之间存在重要关系。

因此，特征工程随着深度学习的发展逐渐从过去关心“如何人工构造特征”，到现在越来越关注“如何构建特征体系以及如何让模型有效地学习特征表示（Feature Representation）”。这种变化也意味着，现代工业级推荐系统中的特征工程与模型结构已经很难完全割裂开来看待。一个特征最终能否发挥作用，不仅取决于特征本身是否包含有效信息，还取决于模型是否能够以合适的方式对这个特征进行表示与交互。例如，同一个 Dense 统计特

征可以直接输入 MLP，也可以基于特征取值分布进行分桶处理或者其他变换；同一个用户历史行为既可以压缩成统计值，也可以保留完整的序列再交给 Transformer 进行建模。不同的选择最终都会影响模型的表达能力、计算成本和线上效果。

总的来说，目前工业级推荐系统中的特征工程更应该被理解为一个从**数据到表征**的完整过程，即：

$$\text{Raw Data} \rightarrow \text{Feature Construction} \rightarrow \text{Feature Representation} \rightarrow \text{Feature Interaction} \rightarrow \text{Prediction} \tag{15.25}$$

其中，Feature Construction 负责从原始数据中提取有价值的信息，Feature Representation 决定这些信息以 Sparse、Dense、Sequence 还是 Embedding 形式进入模型，Feature Interaction 进一步学习不同信息之间的关系，而最终的预测网络则完成具体的 CTR、观看时长、转化率等目标建模。

15.3 行为统计与用户画像

用户历史行为是推荐系统中最重要的信息来源之一。相比年龄、性别、地域等相对稳定的用户属性，用户近期的点击、观看、点赞、收藏、关注和购买等行为能够更加直接地反映用户当前的兴趣。因此，在工业推荐系统中，通常需要对用户历史行为进行统计、聚合和进一步建模，将原始行为转化为能够被排序模型直接使用的行为统计特征或用户画像。

多时间窗口统计是行为统计中最常见的方式之一。针对用户的历史行为，可以分别在近 1 天、7 天、30 天甚至更长时间窗口内进行统计，从而同时刻画用户的短期、中期和长期行为。例如，可以统计用户在不同时间窗口内的点击次数、观看次数、点赞次数、平均观看时长，以及不同 Category、Tag 或者 Cluster 内容的消费占比。不同时间窗口能够提供不同时间尺度的信息，较短窗口更加敏感于用户兴趣变化，而较长窗口则能够反映用户相对稳定的长期偏好。

除了简单的行为次数统计，还可以从**行为频率（Frequency）、行为时效性（Recency）和行为强度（Intensity）**这三个维度描述用户行为。Frequency 用于衡量用户某类行为发生的频率，例如最近 7 天观看体育内容的次数；Recency 用于衡量行为距离当前时间的远近，例如用户最近一次观看体育内容距当前时间的间隔（TimeDiff）；Intensity 则用于描述行为本身的强度，例如观看时长、观看比例、连续观看次数等。通过这三个维度，通常可以比单纯的行为次数更加完整地刻画用户兴趣。在工业级推荐系统的用户兴趣建模中通常也是根据这三个维度来进行特征选择。

在此基础上，我们可以进一步构建**用户兴趣标签**。例如，根据用户历史观看、点击和互动行为，可以为用户生成体育、游戏、美食、娱乐等兴趣标签，并进一步统计不同兴趣的行为次数、行为占比或者兴趣权重。兴趣标签既可以通过人工规则和统计方法生成，也可以通过聚类、Embedding 以及模型自动学习等方式得到。

除此之外，用户兴趣还具有明显的时间演化特征。一个用户可能长期喜欢体育内容，但在最近一段时间突然开始大量消费游戏内容。因此，现代推荐系统通常需要同时建模**长期兴趣和短期兴趣**。长期兴趣主要反映用户相对稳定的消费偏好，短期兴趣则更加关注用户最近一段时间甚至当前 Session 中的行为。二者结合能够同时兼顾兴趣的稳定性和时效性。

更进一步地，一个用户通常并不存在单一兴趣，而可能同时具有多个相互独立的兴趣方向。例如，一个用户可能同时喜欢篮球、游戏和美食。因此，推荐系统还可以进一步采用**多兴趣表示**，将用户表示为多个兴趣向量：

$$E_u = \{e_u^1, e_u^2, \ldots, e_u^K\} \tag{15.26}$$

其中 e_u^k 表示用户的第 k 个兴趣表示。相比单一的 User Embedding，多兴趣表示能够更加细粒度地刻画用户不同兴趣，并可以在召回和排序阶段分别与候选 Item 进行匹配。

总体来看，行为统计与用户画像的核心目标，是将用户大量、复杂且不断变化的历史行为压缩为模型能够有效利用的信息表示。随着序列模型和 Embedding 模型的发展，传统依赖人工规则的用户画像也逐渐向基于深度模型的兴趣表示演进，从简单的统计标签逐步发展为包含长期兴趣、短期兴趣和多兴趣的 Dense 表征。基于上述行为统计与用户画像构建的特征，排序模型能够更准确地理解用户兴趣，从而提升推荐效果。有了上述关

于用户兴趣相关的特征构造，给用户兴趣建模相关的研究奠定了基础。关于用户兴趣建模的研究方法，我们将会在后续的第20章中进行详细的介绍。

15.4 实时特征

用户行为通常具有明显的时效性，尤其是在短视频、直播以及信息流等推荐场景中，用户兴趣可能在很短时间内发生快速的变化。因此，只依赖离线计算得到的静态特征往往无法及时反映用户最新的状态及用户最新的兴趣。因此，工业级推荐系统通常需要根据特征更新的频率，将特征划分为离线特征、准实时特征和实时特征，并进一步引入 Session 特征和多时间尺度特征来引入更多时间尺度下的信息增量。

离线特征通常基于历史全量数据进行批量计算，例如用户近 30 天的观看次数、物品历史 CTR、用户长期兴趣标签等。这类特征数据量较大、计算逻辑相对复杂，但更新频率通常较低，适合通过离线计算任务定期生成。**准实时特征**则介于离线特征和实时特征之间，通常以分钟级、小时级或者其他固定周期进行更新。例如用户最近 1 小时的观看次数、物品最近 1 小时的点击量等。这类特征能够在计算成本和特征时效性之间取得一定平衡。**实时特征**则更加关注用户刚刚发生的行为。例如用户刚刚点击了什么内容、最近连续观看了哪些视频、当前 Session 已经观看了多少个 Item 等。这类信息通常需要通过实时数据流进行计算，并快速同步到线上特征服务，使排序模型能够及时感知用户状态变化。

在实时推荐场景中，**Session 特征**也是比较重要的特征。Session 可以理解为用户当前连续的一段行为过程，其包含的信息通常比长期用户画像更加接近用户当前意图。例如用户在当前 Session 中连续观看多个篮球视频，那么即使用户长期兴趣画像并不突出篮球，模型也应该能够根据当前 Session 行为快速调整推荐结果。因此，Session 特征通常用于补充长期用户画像，增强模型对短期兴趣和即时意图的感知能力。进一步地，工业推荐系统通常会构建**多时间尺度特征**，同时维护长期、中期、短期和实时行为信息：

$$\text{Long-term} + \text{Medium-term} + \text{Short-term} + \text{Real-time} \tag{15.27}$$

其中，长期特征用于描述稳定兴趣，中短期特征用于反映兴趣变化，而实时特征则用于捕捉当前 Session 和即时行为。通过不同时间尺度的特征组合，可以在兴趣稳定性与实时性之间取得更好的平衡。

最后，实时特征还会带来一个重要的工程问题，即 **Training 与 Serving 的一致性问题**。由于模型训练阶段使用的是离线生成的特征，而线上推理阶段使用的是实时特征，如果两者在计算逻辑、更新时间、数据口径或者缺失处理方式上存在差异，就可能产生明显的训练与线上偏差（Training-Serving Skew），导致离线效果与线上效果不一致。因此，工业级特征系统不仅需要关注特征的计算速度，还需要保证离线与在线特征定义、计算逻辑和数据口径的一致性。除此之外，用户行为序列的实时特征产出通常采用流式计算的方式，通过实时数据流处理框架（如 Flink、Spark Streaming 等）对用户行为进行增量计算，并将结果快速写入在线特征存储系统，以供线上模型实时使用。但在离线训练时拿到的这些特征已经是过去延迟的行为序列特征。如果在离线训练环节直接获取最新的实时特征，可能就会存在**“特征穿越”**的问题，即模型在训练时使用了未来的信息，导致训练与线上推理结果的不一致。

整体来看，实时特征的核心并不是单纯追求“更快地更新特征”，而是在特征时效性、计算成本、服务延迟和数据一致性之间进行平衡。随着推荐系统越来越依赖用户实时行为，特征系统也逐渐从传统的离线批处理架构向离线、准实时和实时协同的统一特征体系演进。

15.5 特征交互建模

在传统的推荐系统中，特征交互通常依赖人工设计的特征交叉。例如，将用户的年龄与物品 Category 进行组合，将用户地域与物品 Brand 进行组合，或者进一步构造用户、物品和上下文之间的高阶组合特征。这类方法能够将业务经验显式注入模型，在逻辑斯特回归（LR）、FM 以及 Wide & Deep 等早期推荐模型中发挥了重要的作用。

然而，随着推荐系统中的特征数量不断增加，人工特征交叉也逐渐暴露出明显的问题。一方面，不同特征之

间可能存在大量潜在的交互关系，如果完全依赖人工经验进行组合，很难覆盖所有有效的特征交互；另一方面，随着交叉阶数增加，候选特征组合数量会快速增长，容易产生特征爆炸问题，同时也增加了特征计算、存储和维护成本。此外，人工构造的交叉特征通常具有较强的先验假设，很难根据不同用户、不同物品以及不同推荐场景动态调整特征交互的重要程度。

深度学习的发展使得推荐模型逐渐从“人工设计特征交互”转向“模型自动学习特征交互”。一类典型方法是通过 FM 及其变体自动学习二阶特征交互，另一类方法则通过 Cross Network 显式构造高阶交叉，还有一类方法通过 Attention 机制学习不同特征之间的动态依赖关系。进一步地，Transformer 等结构也被引入推荐系统，通过 Self-Attention 机制对大量特征进行统一建模，使模型能够根据当前样本自适应地选择重要的特征交互。

从特征交互方式来看，现代推荐模型中的特征交互大致可以分为三类。第一类是**显式特征交互**，通过 Cross Network 等结构直接构造特征之间的乘积关系，在保证计算效率的同时对交互阶数进行控制；第二类是**隐式特征交互**，通过 DNN 等网络利用非线性变换自动学习特征之间的复杂关系；第三类是**动态特征交互**，通过 Attention 机制根据当前输入动态计算不同特征之间的重要程度。

因此，特征交互建模的核心问题逐渐从“如何人工构造交叉特征”转变为“如何让模型高效、灵活地学习特征之间的交互关系”。本节将从工业推荐系统中具有代表性的特征交互模型出发，依次介绍 DCN、DCN-V2、AutoInt、FiBiNet、AFN 等方法，分析不同模型在交互阶数、参数规模、表达能力和计算效率之间的权衡。

15.5.1 DCN

DCN 模型源自于 2017 年谷歌与斯坦福大学合作的论文《Deep & Cross Network for Ad Click Predictions》[**wang2017deep**]。DCN 的核心思想是在传统深度神经网络之外引入 Cross Network，通过显式构造输入特征与网络中间表示之间的交叉关系，在较低计算复杂度下学习高阶特征交互。对于推荐模型而言，假设经过 Embedding 层以及其他特征处理之后得到输入向量 $\mathbf{x}_0 \in \mathbb{R}^{d\times 1}$，DCN 模型通过多层 Cross Layer 逐步构造高阶交互特征。第 l 层 Cross Layer 可以表示为：

$$\mathbf{x}_{l+1} = \mathbf{x}_0 \mathbf{x}_l^T \mathbf{w}_l + \mathbf{b}_l + \mathbf{x}_l \tag{15.28}$$

其中，$\mathbf{x}_l$ 表示第 l 层 Cross Layer 的输入，$\mathbf{w}_l \in \mathbb{R}^{d\times 1}$ 表示该层的权重向量，$\mathbf{b}_l \in \mathbb{R}^{d\times 1}$ 表示偏置向量，$\mathbf{x}_0\mathbf{x}_l^T$ 表示输入特征与上一层表示相乘之后的矩阵，最终通过与 $\mathbf{w}_l$ 进行乘积得到交叉项。

DCN 模型最重要的特点在于：其交叉操作始终保留原始输入 $\mathbf{x}_0$，因此随着 Cross Layer 不断堆叠，可以逐层增加特征交互的阶数。假设 $\mathbf{x}_0$ 包含多个原始特征，那么第一层 Cross Layer 主要用于构造二阶交互，第二层可以进一步形成三阶交互，以此类推。因此，当堆叠 L 层 Cross Network 时，模型理论上能够显式建模最高 $L+1$ 阶的特征交互：

$$\text{Order}_{max} = L + 1 \tag{15.29}$$

这种设计使得 DCN 模型能够以非常低的计算和参数成本实现高阶特征交互。与传统 DNN 模型完全依赖非线性变换隐式学习交互关系不同，DCN 模型将特征交叉过程显式地加入网络结构中，因此对于推荐系统中大量具有明确交互关系的 Sparse Embedding 特征具有较好的建模能力。

DCN 模型的另一个重要特点是其参数量与输入维度 d 呈线性关系。对于每一层 Cross Layer，只需要学习一个 d 维权重向量和一个 d 维偏置向量，而不需要构造完整的 $d \times d$ 参数矩阵。因此，其参数规模可以近似表示为 $O(Ld)$。这使得 DCN 模型非常适合工业级推荐系统。在实际推荐模型中，用户、物品、作者、Category 以及 Tag 等 Sparse 特征经过 Embedding 之后通常会形成较高维的输入向量，如果直接采用全连接层进行复杂交互，不仅参数量较大，而且难以明确控制交互阶数，而 DCN 模型能够以相对较低的成本显式学习高阶交互。

DCN 模型通常与 DNN 结构结合形成 Deep & Cross 的架构。Cross 网络负责显式学习特征交互，Deep 网络负责通过多层非线性变换学习更加复杂的隐式关系，两者共同提升模型的表达能力。其整体结构可以表示为：

$$h_{deep} = DNN(\mathbf{x}_0) \tag{15.30}$$

$$h_{cross} = CrossNetwork(\mathbf{x}_0) \tag{15.31}$$

最终将两部分表示进行融合，并输入预测层得到 CTR、CVR 等目标的预测结果：

$$\hat{y} = f([h_{deep}; h_{cross}]) \tag{15.32}$$

其中 $[\cdot;\cdot]$ 表示向量 Concat 操作。通过这种结构，模型同时具备了显式特征交互能力和深度非线性建模能力。

DCN 模型的网络结构如下图15.1所示，在共享的 Embedding 层之上有 Cross 网络和 Deep 网络两个分支，最后再将两个分支的输出拼接到一起通过输出层得到最终的预估值。不过，原始 DCN 模型的 Cross Layer 仍然存在一定的表达能力限制。由于每一层使用的是一个低秩的向量形式进行交叉，其特征交互能力受到参数化方式的限制。当输入特征维度进一步增加，或者需要建模更加复杂的特征关系时，简单的单向量参数化可能无法充分捕捉不同维度之间复杂的交互模式。基于这一问题，后续工作进一步提出了 DCN-V2 模型，通过更加灵活的参数化方式增强 Cross 网络的表达能力。

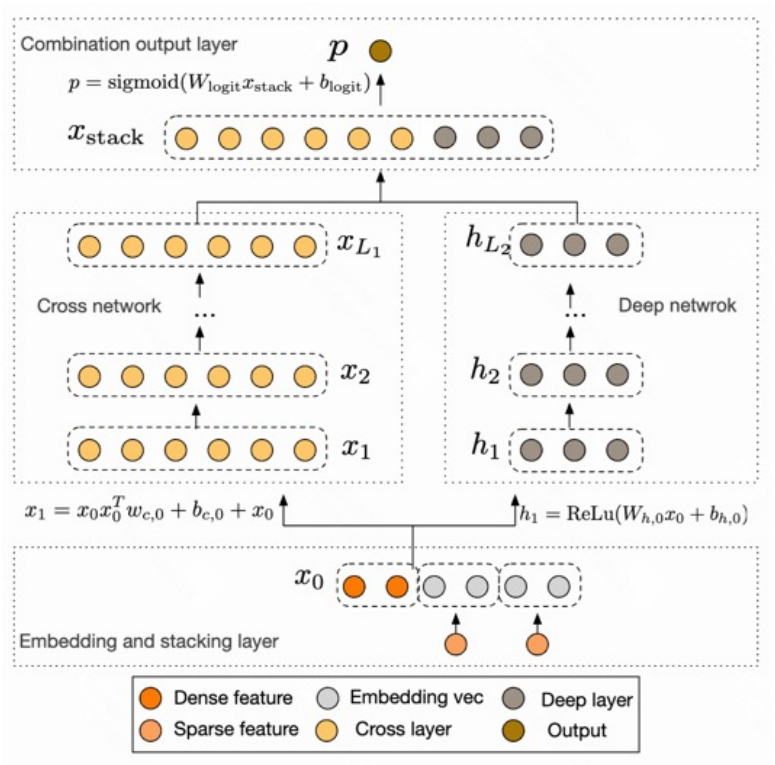


图 15.1: DCN 模型的网络结构示意图

15.5.2 DCN-V2

DCN-V2 模型源自于论文《DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems》[**wang2021dcn**]。DCN-V2 模型的主要改进方向是增强原始 DCN 模型中 Cross 网络这一部分的表达能力，使得模型能够更加灵活地学习不同特征维度之间的交互关系。

原始 DCN 模型中的 Cross Layer 采用向量 w_l 进行参数化，其核心计算可以理解为对 x_0 和 x_l 之间的交互结果进行低秩建模。这种设计具有参数量小、计算效率高的优点，但同时限制了模型的表达能力。DCN-V2 模型将 Cross Layer 进一步扩展为矩阵形式，其基本计算过程可以表示为：

$$\mathbf{x}_{l+1} = \mathbf{x}_0 \odot (\mathbf{W}_l \mathbf{x}_l + \mathbf{b}_l) + \mathbf{x}_l \tag{15.33}$$

其中，$\mathbf{W}_l \in \mathbb{R}^{d \times d}$ 表示可学习的交互矩阵，$\mathbf{b}_l$ 表示偏置项，$\odot$ 表示逐元素乘法。与原始 DCN 模型相比，DCN-V2 模型通过矩阵 $\mathbf{W}_l$ 对不同维度之间的交互进行更加灵活的建模，因此能够表达更加丰富的特征交互模式。

当然，矩阵参数化也会带来更高的参数量和计算成本。对于输入维度 d，完整矩阵 W_l 的参数量达到 $O(d^2)$。当 Embedding 维度较高时，直接使用完整矩阵可能带来较大的计算压力。同时，在 GPU 显存不够的情况下，使用完整参数矩阵时，DCN-V2 模型也可能会出现 OOM 之类的问题。因此，在 DCN-V2 模型的基础上，也可以通

过低秩参数化的方式，将交互矩阵分解为两个较低维度的矩阵，在表达能力和计算效率之间进行权衡。

$$\mathbf{W}_l \approx \mathbf{U}_l \mathbf{V}_l^T \tag{15.34}$$

其中 $\mathbf{U}_l$ 和 $\mathbf{V}_l$ 的维度远小于 d，从而可以在保留较强交互能力的同时降低参数规模。

总的来说，DCN-V2 模型的一个重要工业价值在于，它不再简单追求“更复杂的交叉结构”，而是更加关注不同模型结构在真实大规模 Learning to Rank 系统中的收益与成本。对于工业推荐系统而言，特征交互模型不仅需要关注离线指标，还需要综合考虑参数规模、计算复杂度、线上延迟以及模型稳定性。因此，DCN-V2 模型代表了特征交互模型从理论表达能力进一步向工业可用性演进。

15.5.3 AutoInt

DCN 模型通过显式 Cross Network 构造特征交互，而 AutoInt 模型则采用 Attention 机制让模型自动学习不同特征之间的交互关系。AutoInt 源自北京大学 2019 年 CIKM 会议论文《AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks》[**song2019autoint**]，其核心思想是将每一个特征域的 Embedding 视为一个 Token，通过 Multi-Head Self-Attention 学习不同特征域之间的依赖关系。AutoInt 模型的网络结构如下图15.2所示。

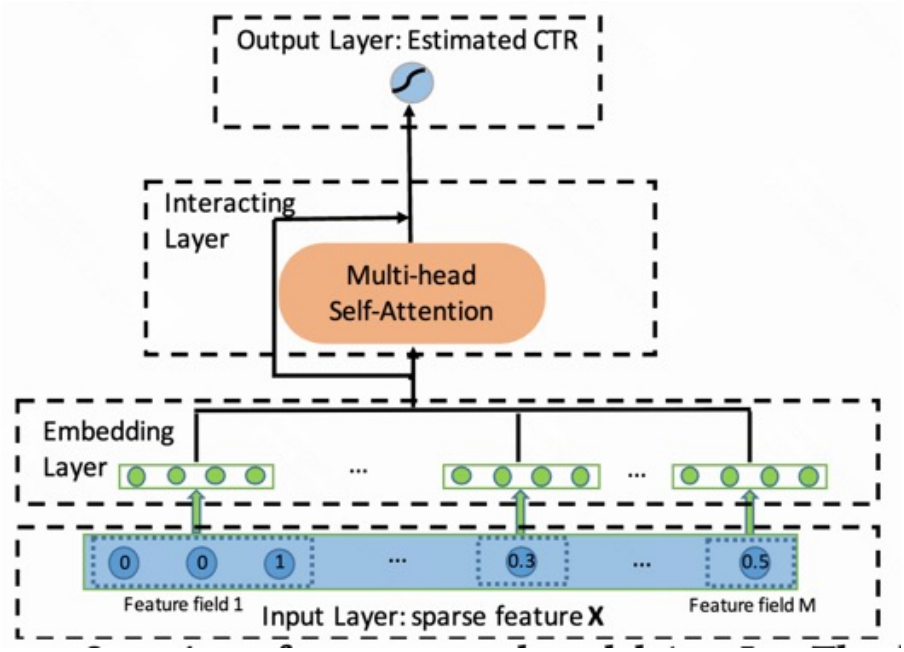


图 15.2: AutoInt 模型的网络结构示意图。

具体来说，AutoInt 模型假设模型包含 n 个特征域，每个特征域经过 Embedding 后得到：

$$E = [e_1, e_2, \ldots, e_n]^T \tag{15.35}$$

对于 Self-Attention 中的一个 Head，可以分别通过三个线性变换得到 Query、Key 和 Value：

$$Q = EW_Q, \quad K = EW_K, \quad V = EW_V \tag{15.36}$$

随后计算不同特征域之间的 Attention 权重：

$$A = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) \tag{15.37}$$

最终得到特征交互后的表示：

$$H = AV \tag{15.38}$$

与 DCN 模型通过显式乘积构造交互不同，AutoInt 模型通过 Attention 权重动态决定哪些特征域之间应该进行重点交互。例如，在一个短视频推荐模型中，用户 ID 与视频 ID、用户兴趣与视频类别以及作者与用户兴趣之间的

重要程度可能并不相同，AutoInt 模型就能够针对不同的样本来动态学习这些特征之间复杂的交互关系。

除此之外，AutoInt 模型还可以通过堆叠多层 Self-Attention 逐步学习更高阶的特征交互。低层 Attention 可以学习较为直接的二阶关系，而随着网络层数增加，上一层已经融合过的特征表示可以进一步参与新的 Attention 计算，从而形成更加复杂的高阶交互。相比起 DCN 模型，AutoInt 模型的主要优势在于其交互关系更加灵活，并且能够通过 Multi-Head Attention 机制从不同子空间学习不同类型的特征关系。但与此同时，Attention 的计算复杂度通常与特征域数量呈平方关系，因此在特征域数量较多的工业推荐模型中，需要结合特征数量、Embedding 维度以及线上延迟要求进行合理设计。

15.5.4 FiBiNet

FiBiNet 模型源自于新浪微博在 RecSys 2019 会议发表的论文《FiBiNET: Combining Feature Importance and Bilinear Feature Interaction for Click-Through Rate Prediction》[**huang2019fibinet**]。传统特征交互模型通常对不同特征采用相同的交互方式，难以体现不同特征在 CTR 预测任务中的重要性差异；同时，FM 中的内积（Inner Product）以及 NFM、AFM 中的 Hadamard Product 等交互方式较为简单，对特征之间更加细粒度的关系建模能力有限。因此，FiBiNet 模型对传统的方法进行了改进。它的核心思想如下：

定义 15.2

FiBiNet 模型分别从特征重要性建模和特征交互建模两个角度进行改进，通过引入 SENET 动态学习不同 Field 的重要性，然后通过 Bilinear Interaction 对特征之间的二阶交互进行更加灵活的建模。

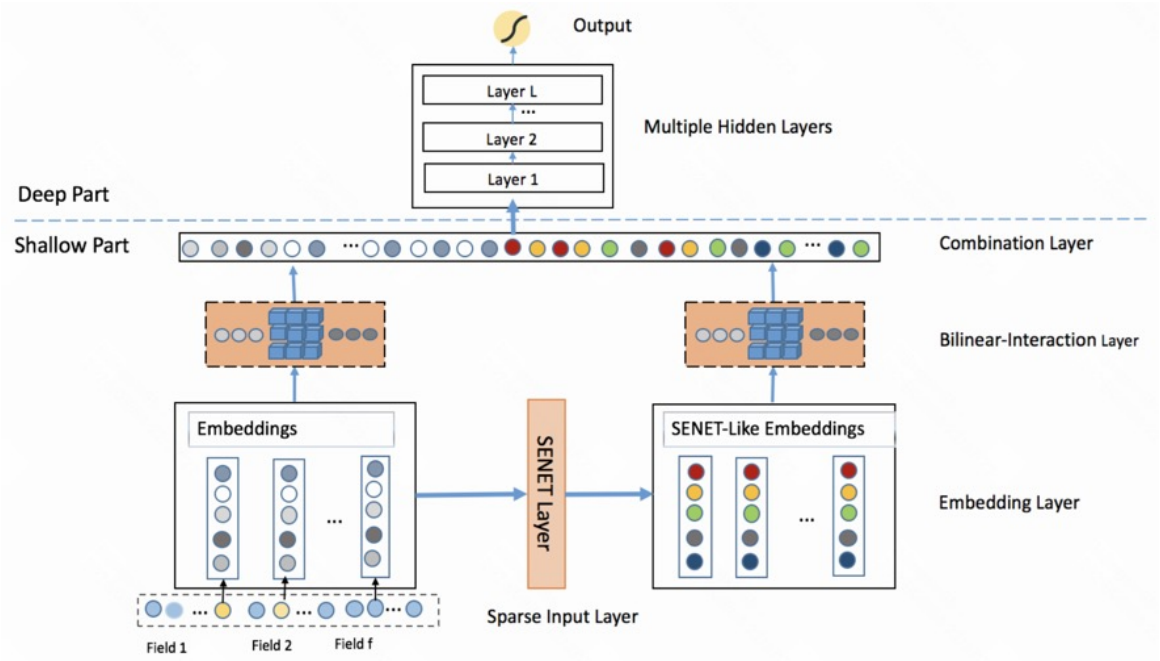


图 15.3: FiBiNet 模型的网络结构示意图。

如图 15.3所示，FiBiNet 模型整体由 Sparse Input Layer、Embedding Layer、SENET Layer、Bilinear-Interaction Layer、Combination Layer、DNN 以及 Output Layer 组成。在给定用户、物品以及上下文等稀疏特征后，FiBiNet 模型首先通过 Embedding Layer 将不同 Field 映射为低维稠密向量，得到原始 Field Embedding E。随后，Embedding 一方面直接输入 Bilinear-Interaction Layer，另一方面经过 SENET Layer 进行特征重要性建模，得到重新加权后的 Embedding V。Bilinear-Interaction Layer 分别对 E 和 V 进行二阶特征交互，得到两组交互特征 P 和 Q；Combination Layer 将两组交互结果进行拼接，并将其输入后续 DNN 以进一步学习高阶特征交互。最终，模型将 DNN 输出与线性部分结合，通过 Sigmoid 函数得到最终的 CTR 预测结果。因此，从整体的网络结构上来看，FiBiNet 模型实

际上形成了两条并行的特征交互路径：

$$E \xrightarrow{\text{Bilinear Interaction}} P, \qquad E \xrightarrow{\text{SENET}} V \xrightarrow{\text{Bilinear Interaction}} Q \tag{15.39}$$

其中，第一条路径直接在原始 Embedding 上进行特征交互，保留原始特征之间的关系；第二条路径则首先根据不同 Field 的重要性对 Embedding 进行动态重加权 (reweighting)，然后再进行特征交互，从而突出更加重要的特征。两条路径的结果最终进行融合，并交由 DNN 完成更高阶的交互建模。

FiBiNet 模型的 Sparse Input Layer 和 Embedding Layer 与 DeepFM[**guo2017deepfm**] 等经典 CTR 模型基本一致。对于用户画像、物品属性、上下文等离散特征，模型首先采用稀疏表示，然后通过 Embedding Layer 将其映射到低维稠密空间。假设输入包含 f 个 Field，则 Embedding Layer 的输出可以表示为：

$$E = [e_1, e_2, \cdots, e_f], \qquad e_i \in \mathbb{R}^k \tag{15.40}$$

其中，e_i 表示第 i 个 Field 的 Embedding，k 表示 Embedding 维度。FiBiNet 模型后续的 SENET 和 Bilinear-Interaction 均以这些 Field Embedding 作为基本输入。FiBiNet 模型借鉴计算机视觉领域 SENET（Squeeze-and-Excitation Network）的思想，对不同 Field 的 Embedding 进行动态加权。其基本假设是：对于特定的 CTR 预测任务，不同特征的重要性并不相同。因此，模型应该能够根据当前输入动态调整不同 Field 的贡献，而不是对所有 Field 进行等权处理。

SENET 主要包含 Squeeze、Excitation 和 Re-Weight 三个步骤。首先，在 Squeeze 阶段，对每个 Field 的 Embedding 进行 Pooling，将其压缩为一个标量统计量。FiBiNet 采用 Mean Pooling：

$$z_i = F_{\text{sq}}(e_i) = \frac{1}{k}\sum_{t=1}^{k} e_i^{(t)} \tag{15.41}$$

从而得到 Field 级别的统计向量：

$$Z = [z_1, z_2, \cdots, z_f] \tag{15.42}$$

随后，在 Excitation 阶段，通过两个全连接层根据 Z 学习各 Field 的重要性权重：

$$A = F_{\text{ex}}(Z) = \sigma_2\left(W_2\sigma_1(W_1 Z)\right) \tag{15.43}$$

其中，$A = [a_1, a_2, \cdots, a_f]$ 表示不同 Field 的权重，W_1 和 W_2 分别对应降维和升维操作，降维比例由超参数 r 控制。最后，在 Re-Weight 的阶段，将 Field 权重作用于原始 Embedding 得到 V：

$$V = [a_1 e_1, a_2 e_2, \cdots, a_f e_f] = [v_1, v_2, \cdots, v_f] \tag{15.44}$$

在经过 SENET 后，FiBiNet 模型能够同时保留原始 Embedding E 和重新加权后的 Embedding V。前者保留原始特征信息，后者则强调了当前 CTR 任务下更加重要的 Field，为后续特征交互提供两种互补的特征表示。

在得到原始 Embedding E 和 SENET Embedding V 后，FiBiNet 模型通过 Bilinear-Interaction Layer 分别对两组 Embedding 进行二阶特征交互。传统 FM 通常采用内积的形式来建模两个 Field 之间的交互：

$$p_{ij} = e_i^{\top} e_j \tag{15.45}$$

这种方式参数量较小，但不同维度之间的交互关系较为固定。FiBiNet 模型进一步引入 Bilinear Interaction，通过可学习的矩阵对 Embedding 进行线性变换，再与另一个 Embedding 进行 Hadamard 乘积，从而获得更加细粒度的交互表示：

$$p_{ij} = (e_i^{\top} W) \odot e_j \tag{15.46}$$

其中，$W \in \mathbb{R}^{k\times k}$，$\odot$ 表示 Hadamard 乘积。FiBiNet 模型根据交互参数的共享方式总共设计了三种不同的 Bilinear Interaction 形式，如下所示：

- **Field-All**：所有 Field 之间共享同一个交互矩阵 W，参数量最小；
- **Field-Each**：每个 Field 拥有独立的交互矩阵 W_i，可以针对不同 Field 学习不同的变换；
- **Field-Interaction**：每一对 Field 拥有独立的交互矩阵 W_{ij}，表达能力最强，但参数量也最高。

对于原始 Embedding E，Bilinear-Interaction Layer 得到交互表示 $P = \{p_{ij} \mid i < j\}$。对于经过 SENET 重新加权后的 Embedding V，采用相同的方式得到 $Q = \{q_{ij} \mid i < j\}$。因此，$P$ 主要描述原始特征之间的二阶交互关系，而

Q 则描述经过特征重要性调整后的二阶交互关系。在得到两组二阶交互特征后，FiBiNet 模型通过 Combination Layer 将 P 和 Q 进行拼接：

$$C = F_{\text{concat}}(P, Q) = [P, Q] \tag{15.47}$$

随后，拼接后的交互特征被输入多层 DNN，通过非线性变换进一步学习高阶特征交互：

$$a^{(l)} = \sigma\left(W^{(l)} a^{(l-1)} + b^{(l)}\right) \tag{15.48}$$

最终，DNN 输出的高阶特征表示与模型的线性部分共同用于 CTR 预测：

$$\hat{y} = \sigma\left(w_0 + \sum_{i=1}^{m} w_i x_i + y_d\right) \tag{15.49}$$

其中，y_d 表示 DNN 部分输出的预测结果。从模型结构来看，FiBiNet 模型实际上将**显式二阶特征交互**与**隐式高阶特征交互**结合起来：Bilinear-Interaction Layer 负责显式建模二阶关系，DNN 则进一步学习高阶非线性交互。同时，SENET 为显式交互增加了 Field 级别的动态重要性建模，使模型能够针对不同 CTR 任务自适应地调整特征贡献。

总体而言，FiBiNet 模型的结构并不复杂，其主要贡献可以概括为两个方面：一是通过 SENET 动态学习特征重要性，二是通过 Bilinear Interaction 增强特征交互。因此，FiBiNet 模型也是较早将“特征重要性”和“特征交互”统一到同一个 CTR 模型中的代表性工作。其后续的研究重点则逐渐从固定的二阶交互进一步发展到更高阶、更灵活以及更加高效的特征交互建模。

15.5.5 AFN

AFN（Adaptive Factorization Network）模型源自于 AAAI 2020 会议的论文《Adaptive Factorization Network: Learning Adaptive-Order Feature Interactions》[**cheng2020adaptive**]。AFN 模型主要针对传统因子分解模型在高阶特征交互建模中的两个问题进行优化。一方面，FM、HOFM（高阶 FM）等模型通常需要预先设定最大交互阶数，阶数越高，潜在的特征组合数量越多，计算和参数开销也随之增加；另一方面，这些方法通常采用“先枚举、后筛选”的方式构造交叉特征，大量无效的特征组合可能引入噪声。AFN 的核心思想如下：

定义 15.3

AFN 模型不再显式枚举不同阶数的交叉特征，而是通过学习每个特征在交叉组合中的幂，从数据中自适应地学习有用的任意阶特征交互。

AFN 模型的网络结构如图 15.4所示，模型首先将输入的稀疏特征映射为低维 Embedding，随后将不同 Field 的 Embedding 输入对数变换层。该层由多个向量级的对数神经元组成，每个神经元对应一种潜在的特征组合，并通过可学习的参数决定各个 Field 在该组合中的幂。经过指数变换后得到自适应阶数的交叉特征，再通过若干全连接层进一步学习元素级的非线性交互，最终输出预测结果。

从整体结构来看，AFN 可以概括为：

$$\text{Embedding} \rightarrow \text{对数变换} \rightarrow \text{指数变换} \rightarrow \text{隐藏层} \rightarrow \hat{y} \tag{15.50}$$

其中，前半部分负责显式学习不同阶数的交叉特征，后半部分则通过神经网络进一步建模这些交叉特征之间的非线性关系。与 FM/HOFM 直接指定二阶或高阶交互不同，AFN 的交互阶数由模型训练过程中学习得到。论文中的实验也表明，不同 Logarithmic Neuron 最终学习到的交叉特征阶数可以覆盖较宽的范围，而不是固定为预设的某一个阶数。

AFN 模型中的核心组件是对数变换层（Logarithmic Transformation Layer），其基本思想来源于对数神经网络（Logarithmic Neural Network，LNN）。对于输入 x，LNN 中的对数神经元可以表示为：

$$y = \exp\left(\sum_i w_i \ln x_i\right) = \prod_i x_i^{w_i} \tag{15.51}$$

其中，w_i 为可学习参数。由于对数变换能够将乘法转换为加法，因此上述结构可以通过学习 w_i 直接控制不同输

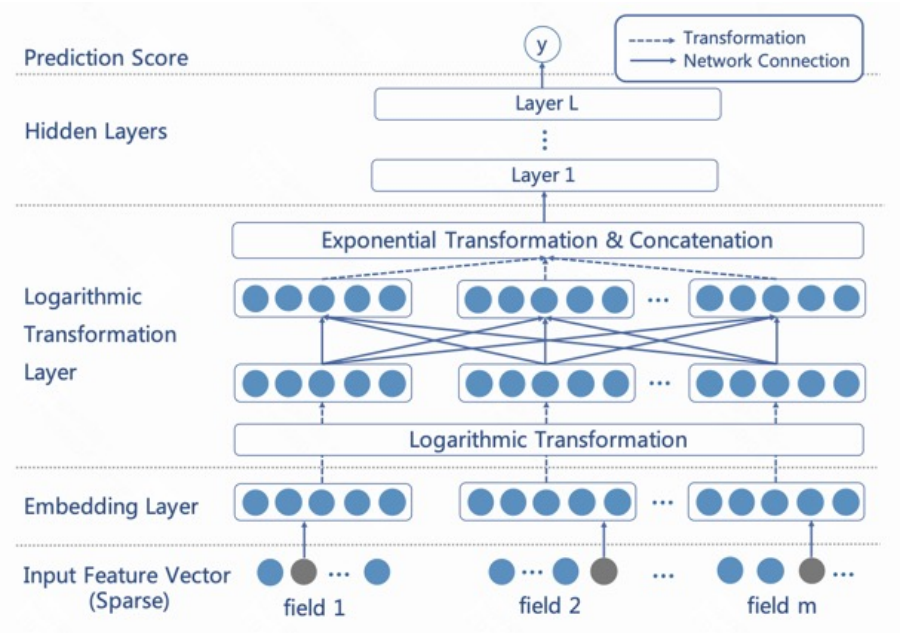


图 15.4: AFN 模型的网络结构示意图

入特征在最终乘积中的幂。AFN 正是利用这一性质，将 w_i 解释为特征在交叉组合中的阶数，从而避免显式枚举不同阶数的交叉特征。

对于 CTR 任务中的 Field Embedding，假设模型输入中包含 m 个 Field：

$$E = [e_1, e_2, \cdots, e_m] \qquad e_i \in \mathbb{R}^k \tag{15.52}$$

其中，e_i 表示第 i 个 Field 的 Embedding，k 表示 Embedding 维度。AFN 模型中的对数神经元以向量级方式作用于这些 Embedding，并为每个 Field 学习一个幂参数。对于第 j 个对数神经元，可以表示为：

$$z_j = \exp\left(\sum_{i=1}^{m} w_{ij} \ln(e_i)\right) \tag{15.53}$$

其中，w_{ij} 表示第 i 个 Field 在第 j 个交叉特征中的学习权重。在实际计算中，为了保证对数变换和梯度传播的稳定性，AFN 模型对 Embedding 进行了相应的数值处理，并在对数变换和指数变换等位置引入 Batch Normalization。直观来看，如果某个 Field 对应的 w_{ij} 接近 0，则该 Field 对当前交叉特征的贡献较小；如果多个 Field 对应的 w_{ij} 具有较大的绝对值，则它们将共同构成更加复杂的高阶交互。因此，一个对数神经元实际上可以看作是在自动搜索一种特征组合及其交互阶数。

AFN 模型与传统 FM/HOFM 模型最大的区别在于交互阶数不再由人工预先指定。以两个 Field 为例，如果对应的学习参数分别为 $w_1 = 1$ 和 $w_2 = 1$，则可以得到类似于 $e_1 e_2$，即传统的二阶交互；如果三个 Field 对应的参数均接近 1，则可以得到类似 $e_1 e_2 e_3$，即三阶交互。而 AFN 中的参数并不要求只能取 0 或 1，因此还可以学习更加灵活的非整数阶组合。

对于一个对数神经元，其对应交叉特征的整体阶数可以理解为各个特征阶数的绝对值之和：

$$O_j = \sum_{i=1}^{m} |w_{ij}| \tag{15.54}$$

因此，模型无需预先规定“只学习二阶”或“最多学习三阶、四阶”等规则，而是通过训练自动得到不同神经元对应的有效交互阶数。需要注意的是，AFN 模型并不是简单地将所有可能的高阶特征组合全部计算出来，而是通过有限数量的对数神经元主动学习具有区分性的交叉特征。因此，AFN 模型的核心并非“暴力增加交互阶数”，而是用可学习的特征阶数替代人工设定的交互阶数和显式特征枚举。

在经过 Log 变换和指数变换后，AFN 模型得到一组自适应阶数的交叉特征。随后，这些特征被输入多层全

连接网络，通过 ReLU 等非线性函数进一步学习元素级的高阶交互：

$$z^{(l)} = \text{ReLU}\left(W^{(l)} z^{(l-1)} + b^{(l)}\right) \tag{15.55}$$

这里的 DNN 与 AFN 模型前面的 Log 变换承担不同的作用：Log 变换主要负责显式发现不同阶数的交叉特征，而后续 DNN 则进一步学习这些交叉特征之间的非线性组合，从而增强模型的表达能力。最终，通过预测层得到 CTR 预测结果：

$$\hat{y} = w_p^\top z^{(L)} + b_p \tag{15.56}$$

对于二分类 CTR 预测任务，可以进一步将预测结果输入 Sigmoid 函数，并采用 Log loss 进行优化。

总体而言，AFN 的核心可以概括为**“通过 Log 变换学习特征阶数，从而自适应地构造任意阶特征交互”**。相比起 FM/HOFM 等预先确定交互阶数、再枚举相应特征组合的方式，AFN 模型可以将交互阶数本身纳入模型参数进行学习，在控制模型规模的同时能够发现不同阶数的有效交叉特征。因此，AFN 模型也可以看作是从传统**固定阶数的特征交互**向**自适应阶数的特征交互**演进的重要代表模型。

15.5.6 FED

FED 模型源自于京东在 CIKM 2020 会议的论文《Dimension Relation Modeling for Click-Through Rate Prediction》[**zhao2020dimension**]。该方法从 Embedding 维度关系的角度重新审视特征交互问题，提出了 **Dimension Relation Module（DRM）**的模块来通过自注意力机制在样本级别学习不同维度之间的隐式关系，并在此基础上进一步构建 Field-wise 和 Element-wise 两类特征交互模块，最终形成了完整的 FED（**F**ield-wise and **E**lement-wise embedding methods with **D**RM）网络。

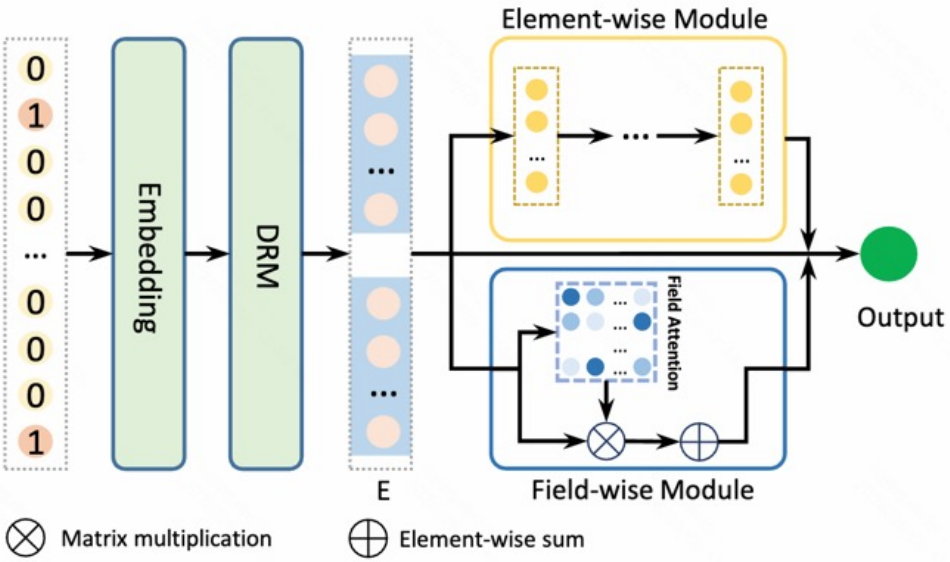


图 15.5: FED 模型的整体网络结构示意图。

FED 模型的整体网络结构如图15.5所示。模型首先将输入的稀疏特征映射为 Embedding 矩阵 V，随后经过 DRM 显式建模不同 Embedding 维度之间的关系，得到增强后的 Embedding 表示 E。在此基础上，模型分别从 Field-wise 和 Element-wise 两个粒度进行特征交互建模：前者利用注意力机制建模不同原始特征域之间的关系，后者通过 DNN 学习更加细粒度的非线性特征交互。最终，将三部分表示进行拼接，并通过 sigmoid 激活的线性层得到 CTR 预测结果。

具体来说，对于一个包含 m 个 Field 的输入样本，将第 i 个 Field 经过 Embedding 映射得到 d 维稠密向量：

$$\mathbf{v}_i = [v_{i,1}, v_{i,2}, \cdots, v_{i,d}]^T \in \mathbb{R}^d \tag{15.57}$$

将所有 Field 的 Embedding 进行拼接，可以得到 Embedding 矩阵：

$$V = [\mathbf{v}_1, \mathbf{v}_2, \cdots, \mathbf{v}_m]^T \in \mathbb{R}^{m \times d} \tag{15.58}$$

传统特征交互方法通常以 Field 为基本单位进行建模，即关注 $\mathbf{v}_i$ 与 $\mathbf{v}_j$ 之间的关系。FED 模型进一步从另一个角度观察 Embedding 空间，将 Embedding 矩阵转置：

$$U = V^T \in \mathbb{R}^{d \times m} \tag{15.59}$$

其中，第 j 个向量：

$$\mathbf{u}_j = [v_{1,j}, v_{2,j}, \cdots, v_{m,j}]^T \tag{15.60}$$

表示所有 Field 在第 j 个 Embedding 维度上的响应。因此，$\mathbf{v}_i$ 描述一个 Field 在不同隐式维度上的表示，而 $\mathbf{u}_j$ 则描述不同 Field 在同一个隐式维度上的行为。虽然这些 Embedding 维度不像原始 Field 那样具有明确的语义，例如“性别”或“地域”，但不同维度之间同样可能存在具有预测价值的关联关系。FED 模型的 DRM 模块正是针对这一关系进行显式建模。

为了建模不同 Embedding 维度之间的关系，FED 模型提出了 **Dimension Relation Module（DRM）**。其结构如图15.6所示。DRM 首先将 Embedding 矩阵转置得到维度特征矩阵 U，然后通过 Self-Attention 计算不同 Embedding 维度之间的相关性，形成维度级 Attention Map，最后利用该 Attention Map 重新聚合原始 Embedding，并通过残差连接得到增强后的 Embedding 表示。

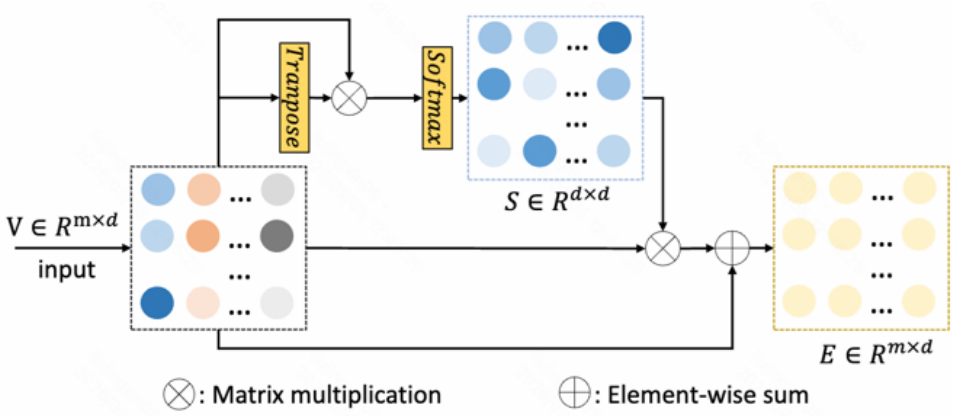


图 15.6: DRM 模块的网络结构示意图。

在 DRM 中，首先对维度特征 U 进行两个线性变换，并通过内积计算不同维度之间的相关性：

$$S = \text{softmax}\left((UW_\theta)(UW_\phi)^T\right) \tag{15.61}$$

其中，$W_\theta, W_\phi \in \mathbb{R}^{m \times m}$ 为可学习的变换矩阵，$S \in \mathbb{R}^{d \times d}$ 为维度级 Attention Map。S_{ji} 可以理解为第 i 个 Embedding 维度对第 j 个 Embedding 维度的重要性，即不同隐式维度之间的相互影响关系。在得到维度关系后，利用 Attention Map 对原始 Embedding 进行信息聚合：

$$E = (W_\delta V)S \tag{15.62}$$

其中，$W_\delta \in \mathbb{R}^{m \times m}$ 用于对原始 Embedding 进行变换。为了避免 Attention 变换破坏原始特征信息，同时改善网络优化，引入类似 ResNet 的残差连接，最终得到：

$$E = (W_\delta V)S + V \tag{15.63}$$

因此，DRM 的本质如下：

定义 15.4

DRM 根据当前样本中不同 Embedding 维度之间的相关性，对原始 Embedding 进行自适应的信息重组。相比直接使用原始 Embedding，增强后的 E 同时融合了其他维度的信息，从而为后续的 Field-wise 和 Element-wise 交互提供更加丰富的特征表示。

在获得 DRM 增强后的 Embedding E 后，FED 模型进一步从 Field 粒度建模显式的特征交互。其核心思想与 DRM 类似，但 Attention 的作用对象从 Embedding 维度变成了原始 Field。首先对 E 进行线性变换，并计算不同

Field 之间的相关性：

$$H = \text{softmax}\left((EW_\theta^1)(EW_\phi^1)^T\right) \tag{15.64}$$

其中，$W_\theta^1, W_\phi^1 \in \mathbb{R}^{d\times d}$，$H \in \mathbb{R}^{m\times m}$ 为 Field 级 Attention Map。H_{ji} 表示第 i 个 Field 对第 j 个 Field 的重要程度。随后利用 Field Attention 对 Embedding 进行信息聚合，并加入残差连接：

$$F = H(EW_\delta^1) + E \tag{15.65}$$

其中，$W_\delta^1 \in \mathbb{R}^{d\times d}$ 为可学习的变换矩阵。经过该操作后，每个 Field 的表示不仅包含自身信息，还融合了其他 Field 的信息，因此可以将其视为一种显式的 Field-level 特征交互。通过堆叠多个 Field-wise 层，还可以逐步构造更高阶的组合特征。最终，将最后一层输出 F 进行向量化，得到 Field-wise 表示 $\mathbf{x}_{field}$。

除了 Field 级别的显式交互外，FED 模型还通过 Element-wise Module 从更加细粒度的 Embedding 元素层面学习隐式非线性特征交互。具体而言，将 DRM 输出的增强 Embedding E 直接展开为一维向量，并输入多层 DNN。对于第 l 层隐藏层，其计算过程为：

$$\mathbf{h}^{l+1} = f\left(W_l\mathbf{h}^l + \mathbf{b}_l\right) \tag{15.66}$$

其中，W_l 和 $\mathbf{b}_l$ 分别表示第 l 层的权重矩阵和偏置，$f(\cdot)$ 采用 ReLU 激活函数。最终隐藏层输出经过向量化得到 Element-wise 表示 $\mathbf{x}_{element}$。与 Field-wise Module 通过 Attention 显式建模 Field 之间的关系不同，Element-wise Module 不显式指定特征交互结构，而是依靠 DNN 的非线性变换自动学习 Embedding 元素之间的隐式组合关系。因此，两者分别从显式和隐式两个角度对特征交互进行补充。

FED 模型最终将 DRM、Field-wise 和 Element-wise 三个模块产生的表示进行拼接：

$$\mathbf{x} = [\mathbf{x}_{dim}; \mathbf{x}_{field}; \mathbf{x}_{element}] \tag{15.67}$$

其中，$\mathbf{x}_{dim}$ 为 DRM 输出的向量化表示，$\mathbf{x}_{field}$ 为 Field-wise Module 的输出，$\mathbf{x}_{element}$ 为 Element-wise Module 的输出。随后通过 Logistic 层得到 CTR 预测结果：

$$\hat{y} = \sigma\left(\mathbf{x}^T\mathbf{w}_{logistic}\right) \tag{15.68}$$

其中，$\sigma(\cdot)$ 为 Sigmoid 函数。对于二分类 CTR 预测任务，模型采用 Log Loss 进行优化：

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right] \tag{15.69}$$

总体来看，FED 模型的核心贡献并不是设计一种新的传统 Field 交叉算子，而是进一步挖掘 Embedding 空间中的维度关系（Dimension Relation）。其中，DRM 从**维度级**建模不同隐式维度之间的关系，Field-wise Module 从**Field 级**建模不同原始特征域之间的关系，而 Element-wise Module 则利用 DNN 学习**元素级**的隐式非线性交互。三者共同构成了从 Dimension、Field 到 Element 的多粒度特征交互建模框架。

15.5.7 CAN

CAN（Co-Action Network）源自于阿里巴巴在 WSDM 2022 会议上的论文《CAN: Feature Co-Action for Click-Through Rate Prediction》[**bian2020can**]。与 DCN 模型通过显式 Cross Layer 构造特征交互、AutoInt 模型通过 Self-Attention 学习特征域之间的依赖关系不同，CAN 模型提出了一种更加灵活的 **Feature Co-Action** 机制。CAN 模型的核心思想如下：

定义 15.5

CAN 模型对于一组需要进行交互的特征，不再直接为不同特征组合学习独立的交互参数，而是让其中一侧特征生成一个轻量级的 Micro-MLP 参数，另一侧特征作为该 Micro-MLP 的输入，从而使两个特征共同决定最终的交互结果。

CAN 模型的这种设计可以理解为一种“特征控制网络”的思想。在传统的 DNN 中，网络参数通常与输入特征相互独立，所有样本共享相同的网络参数；而 CAN 模型允许一侧特征根据自身的 Embedding 动态生成 Micro-

MLP 的 Weight 和 Bias，使得不同的特征 pair 实际上可以对应不同的计算函数。因此，CAN 模型不仅能够建模特征之间的交互关系，还能够让不同特征组合拥有更加独立的参数化方式。可以说，CAN 模型的这种根据特征动态生成网络 weight 和 bias 的机制，也属于一种“个性化网络参数”的方法。关于个性化网络参数方法的更多介绍与讨论，我们将会在后续第24章中进行详细说明。

CAN 模型的整体架构如图15.7所示。对于用户特征集合 U 和目标物品特征集合 M，CAN 模型首先分别通过 Embedding Layer 将 Sparse 特征映射到 Dense Embedding 空间，得到用户侧和物品侧的 Embedding 表示，并通过 Concat 形成基础的用户表示和物品表示。同时，CAN 模型从用户特征和物品特征中分别选择适合作为 Feed Side（输入供给侧）和 Induction Side（参数生成侧）的特征，并通过 Co-Action Unit 进一步进行细粒度的特征交互建模。

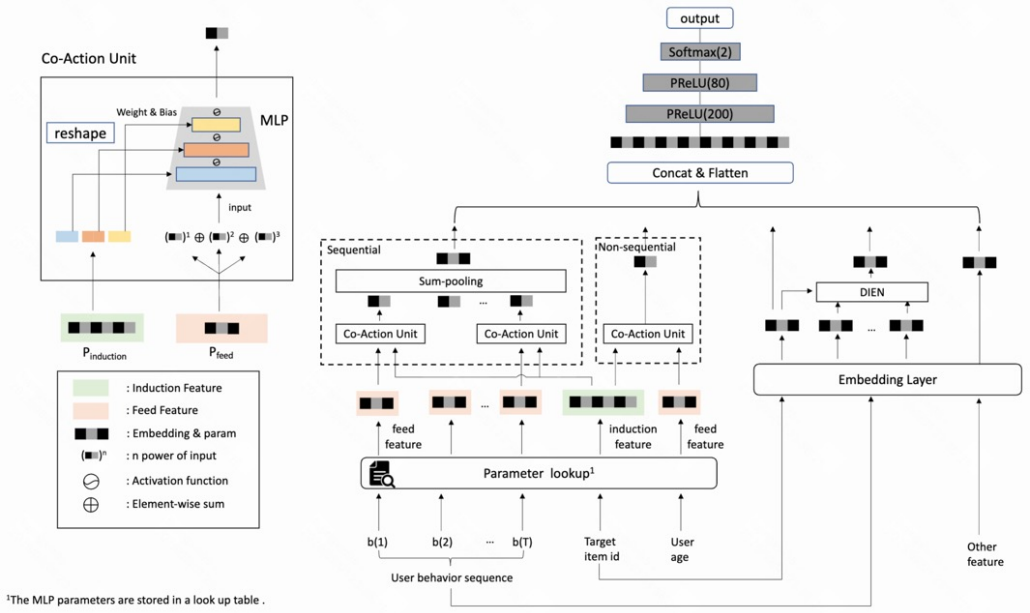


图 15.7: CAN 模型的网络结构示意图

具体来说，假设用户侧原始特征集合为 U，物品侧原始特征集合为 M，经过 Embedding 层之后，可以分别得到：

$$E_U = E(u_1), E(u_2), \ldots, E(u_I), \quad E_M = E(m_1), E(m_2), \ldots, E(m_J) \tag{15.70}$$

随后，将同一侧的 Embedding 进行 Concat，得到用户侧和物品侧的基础表示：

$$e_{user} = Concat(E(u_1), E(u_2), \ldots, E(u_I)), \quad e_{item} = Concat(E(m_1), E(m_2), \ldots, E(m_J)) \tag{15.71}$$

这些基础 Embedding 表示与 Co-Action Unit 产生的特征交互表示共同输入后续 DNN，最终得到 CTR 等目标的预测结果：

$$\hat{y} = DNN(e_{user}, e_{item}, F(u_i, m_j)) \tag{15.72}$$

其中，$F(u_i, m_j)$ 表示用户特征 u_i 与物品特征 m_j 经过 Co-Action Unit 之后得到的交互表示。

CAN 最核心的模块是 **Co-Action Unit**。对于一个具体的特征 Pair，CAN 并不是简单地将两个特征 Embedding 进行 Concat 或者 Inner Product，而是将两个特征分别赋予不同的角色。一侧作为 **Induction Side**，负责生成 Micro-MLP 的网络参数；另一侧作为 **Feed Side**，负责向这个 Micro-MLP 提供输入。假设 Induction Side 对应特征的 Embedding 为 $P_{induction}$，Feed Side 对应特征的 Embedding 为 P_{feed}，则可以表示为：

$$F(u_i, m_j) = H(P_{induction}, P_{feed}) \tag{15.73}$$

其中，$H(\cdot)$ 表示 Co-Action Unit 的计算过程。对于 Induction Side，CAN 模型首先通过 Parameter Lookup 获得一组可学习参数 $P_{induction}$。这些参数并不是传统意义上的 Embedding 表示，而是进一步被 reshape 并切分为 Micro-MLP

每一层的 Weight 和 Bias。假设 Micro-MLP 包含 L 层，则可以表示为：

$$P_{induction} = (w_0 \| b_0) \| (w_1 \| b_1) \| \cdots \| (w_{L-1} \| b_{L-1}) \tag{15.74}$$

其中，w_i 和 b_i 分别表示第 i 层 Micro-MLP 的 Weight 和 Bias，$\|$ 表示 Concat 操作。因此，Induction Side 的参数实际上决定了 Micro-MLP 本身的结构和参数。与之对应，Feed Side 则作为 Micro-MLP 的输入。首先令：

$$h_0 = P_{feed} \tag{15.75}$$

随后，将 Induction Side 生成的参数用于逐层计算：

$$h_i = \sigma(w_{i-1} \otimes h_{i-1} + b_{i-1}), \quad i = 1, 2, \ldots, L \tag{15.76}$$

其中，$\otimes$ 表示矩阵乘法，σ 表示激活函数。最终，CAN 模型将 Micro-MLP 各层的输出进行 Concat，形成特征交互表示 $F(u_i, m_j)$，即：

$$F(u_i, m_j) = Concat(h_1, h_2, \ldots, h_L) \tag{15.77}$$

因此，CAN 模型实际上实现了一个**由特征生成参数、再由参数作用于另一侧特征**的动态交互过程。对于不同的 Induction 特征，Micro-MLP 的参数不同；而对于不同的 Feed 特征，同一个 Induction Side 生成的 Micro-MLP 又可以作用于不同的输入。这使得特征交互从传统的固定函数变成了由特征 Pair 共同决定的动态计算过程。

CAN 模型的这种设计与直接学习特征笛卡尔积具有明显不同。假设用户侧和物品侧分别存在 N 个 ID，如果直接为所有用户 ID 和物品 ID 组合建立独立的交互 Embedding，那么需要学习的参数规模大约为 $O(N^2 D)$，其中 D 为 Embedding 维度。当 ID 规模达到百万甚至更高时，这种笛卡尔积形式的参数规模几乎无法接受。CAN 模型则通过 Induction Side 和 Feed Side 的组合方式避免显式存储所有特征 Pair 的参数。假设 Induction Side 需要 D' 维参数，而 Feed Side 的 Embedding 维度为 D，则整体参数规模可以降低为 $O(N(D' + D))$。因此，CAN 模型不仅能够显著减少参数规模，还能够降低线上系统的存储和计算压力。

在实际推荐系统中，Induction Side 和 Feed Side 并不存在绝对固定的角色。理论上，任意一侧的特征都可以用于生成 Micro-MLP 参数，而另一侧的特征作为 Micro-MLP 输入。实际应用中通常会根据特征规模和数据分布选择更加合适的一侧作为 Induction Side。例如，在广告和推荐场景中，候选 Item 的数量通常远小于用户历史行为中包含的 Item 数量，因此可以将目标 Item 侧特征作为 Induction Side，将用户侧行为特征作为 Feed Side，从而减少需要生成和访问的 Micro-MLP 参数规模。

CAN 模型还可以自然地应用于用户行为序列。假设用户历史行为序列表示为：

$$P_{seq} = \left\{ P_b^{(1)}, P_b^{(2)}, \ldots, P_b^{(T)} \right\} \tag{15.78}$$

其中，$P_b^{(t)}$ 表示用户第 t 次历史行为对应的 Embedding。CAN 模型可以将 Induction Side 生成的 Micro-MLP 分别作用于历史行为中的每一个特征，然后通过 Pooling 得到整体的行为交互表示：

$$H(P_{induction}, P_{seq}) = \sum_{t=1}^{T} H(P_{induction}, P_b^{(t)}) \tag{15.79}$$

这种设计使 CAN 模型不仅可以处理普通的用户特征与物品特征交互，也能够用于建模目标物品与用户历史行为之间的关系，从而将用户兴趣信息直接融入特征交互过程。

除了基础的 Co-Action Unit 之外，CAN 模型还进一步提出了 **Multi-order Enhancement** 机制。基础 Co-Action Unit 主要建立在一阶特征表示之上，但特征交互本身可能存在更加复杂的高阶关系。虽然 Micro-MLP 中的非线性激活已经具备一定的高阶拟合能力，但直接让模型从稀疏数据中学习高阶关系可能存在较大的优化难度。因此，CAN 模型显式引入不同阶数的 Feed 特征作为 Micro-MLP 输入。假设 P_{feed} 为 Feed Side 表示，最多考虑 C 阶交互，则可以表示为：

$$H_{Multi-order}(P_{induction}, P_{feed}) = \sum_{c=1}^{C} H(P_{induction}, (P_{feed})^c) \tag{15.80}$$

其中，$(P_{feed})^c$ 表示 Feed 特征的 c 阶信息，C 表示最大的交互阶数。为了避免高阶计算过程中数值快速增长带

来的训练不稳定问题，CAN 模型采用 Tanh 等有界激活函数对高阶信息进行约束。Multi-order Enhancement 机制的作用并不是简单增加更多网络层数，而是通过显式提供不同阶数的输入，让模型能够更加容易地学习高阶特征交互。相比完全依赖深层 DNN 隐式拟合高阶关系，这种方式可以在不显著增加额外计算和存储成本的情况下增强模型的非线性表达能力。

从整体上来看，CAN 模型与 FM、DCN 等传统特征交互模型之间存在明显的联系。FM 可以看作 CAN 模型的一种特殊情况。当 Micro-MLP 仅包含一层线性变换，并且不使用 Bias 和 Activation 时，Co-Action Unit 可以退化为简单的 Embedding 内积或者线性特征交互，因此 FM 可以被视为 CAN 在特定参数化方式下的一种特殊形式。

CAN 模型的核心价值并不只是“增加一个特征交互模块”，而是改变了特征交互的参数化方式。传统模型通常是**特征表示 + 固定交互函数**，例如直接进行内积、Cross 网络或者 Attention 网络的计算，而 CAN 模型则进一步变成**特征条件化交互函数**，即由一侧特征来动态决定另一侧特征进行交互时所使用的计算函数。这一思想对于工业推荐系统尤其具有价值。推荐模型中的用户、物品、作者、类别、标签以及行为序列等特征域之间并不是完全对称的，不同类型特征之间的交互也具有不同的重要程度。CAN 模型通过 Induction Side 和 Feed Side 的角色划分，使得模型能够以较低的参数规模学习更加细粒度的特征交互，同时避免直接构造大规模特征笛卡尔积所带来的存储和计算问题。

总体而言，CAN 模型代表了推荐系统中特征交互建模从**固定交互函数**向**特征条件化交互函数**发展的一个重要方向。DCN 模型通过显式 Cross Layer 解决高阶交互问题，AutoInt 模型通过 Attention 动态选择重要的特征域关系，而 CAN 模型则进一步让特征本身参与交互函数的生成，使不同特征组合能够拥有更加独立和灵活的交互能力。

15.5.8 FINT

FINT 模型源自于爱奇艺在 ICASSP 2022 会议的论文《FINT: Field-Aware Interaction Neural Network for CTR Prediction》[**zhao2021fint**]。与传统特征交互方法将不同 Field 的 Embedding 直接进行聚合不同，FINT 强调在特征交互过程中保留 Field 边界，为每个 Field 维护独立的交互表示，从而在进行高阶特征交互的同时保留原始 Field 信息，并进一步利用 DNN 学习非线性的高阶特征关系。

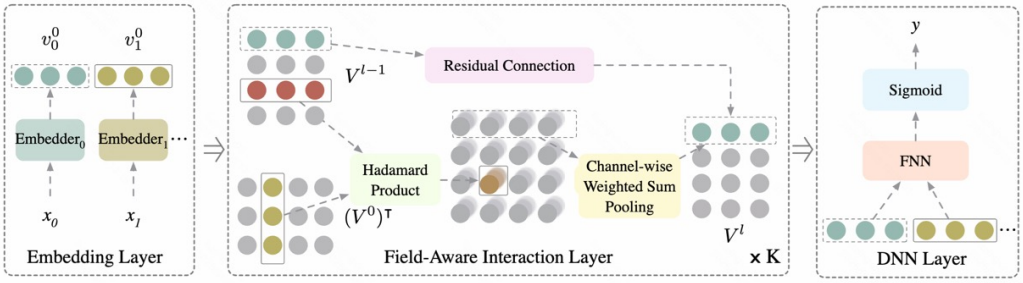


图 15.8: FINT 模型网络结构。

FINT 模型的网络结构如图15.8所示。整体上，FINT 模型由 Embedding Layer、Field-aware Interaction Layer 和 DNN Layer 三个部分组成。模型首先将输入特征映射为稠密 Embedding；随后通过多层 Field-aware Interaction Layer 逐步进行特征交互，每一层通过 Hadamard Product 构造 Field 之间的组合信息，并利用 Field 级别的可学习权重进行聚合，同时通过残差连接保留上一层表示；最后将各 Field 的最终表示进行拼接，并输入 DNN 进一步学习非线性高阶特征交互，输出最终的 CTR 预测结果。

具体来说，假设输入样本包含 M 个特征 Field：

$$X = \{x_0, x_1, \cdots, x_{M-1}\} \tag{15.81}$$

对于每个特征 x_i，首先通过 Embedding 层将其映射为 D 维稠密向量：

$$\mathbf{v}_i^0 \in \mathbb{R}^D \tag{15.82}$$

其中上标 0 表示初始 Embedding 表示。当某个 Field 包含多个特征时，可以对该 Field 内多个特征的 Embedding 进行 sum pooling，然后得到对应的 Field Embedding。我们将所有 Field 的 Embedding 进行组合，可以得到：

$$V^0 = [\mathbf{v}_0^0, \mathbf{v}_1^0, \cdots, \mathbf{v}_{M-1}^0]^T \tag{15.83}$$

与后续交互层生成的表示相比，V^0 始终作为原始 Embedding 参与每一层的特征交互。

Field-aware Interaction Layer 是 FINT 模型的核心模块，其主要目标是在进行高阶特征交互时保持不同 Field 的独立表示。FINT 模型堆叠 K 层 Field-aware Interaction Layer，每一层主要包含两个步骤：Hadamard 乘积和 Field-wise 加权求和。对于第 l 层，首先将上一层表示 $\mathbf{v}_i^{l-1}$ 与初始 Embedding $\mathbf{v}_j^0$ 进行逐元素乘积：

$$\mathbf{v}_i^{l-1} \odot \mathbf{v}_j^0 \tag{15.84}$$

其中 $\odot$ 表示 Hadamard 乘积。随后对所有 Field 产生的交互结果进行加权聚合：

$$\mathbf{v}_i^l = \sum_{j=0}^{M-1} w_{i,j}^l \left(\mathbf{v}_i^{l-1} \odot \mathbf{v}_j^0\right) + u_i^l \mathbf{v}_i^{l-1} \tag{15.85}$$

其中，$w_{i,j}^l$ 表示第 j 个 Field 对第 i 个 Field 的交互权重，u_i^l 为残差连接对应的可学习标量。这里的关键在于，FINT 模型并没有像传统 FM、NFM 等方法一样，在交互过程中将所有 Field 的信息直接压缩成一个统一的向量，而是始终维护每个 Field 对应的交互向量 $\mathbf{v}_i^l$。因此，第 l 层的 $\mathbf{v}_i^l$ 不仅包含 Field i 自身的信息，还融合了其他 Field 的信息。

由于每经过一层都会与初始 Embedding V^0 再次进行 Hadamard 乘积，因此随着层数增加，可以逐步构造更高阶的特征交互。例如，第一层主要形成二阶交互，第二层可以进一步形成三阶交互，以此类推。通过堆叠 K 层，可以在不显式枚举所有高阶组合的情况下逐步学习高阶特征关系。为了避免深层交互过程中的训练退化，FINT 模型在每个 Interaction Layer 中加入了残差连接，使上一层的 Field 表示能够直接传递到下一层。经过 K 层 Field-aware Interaction Layer 后，得到最终的 Field 表示：

$$V^K = [\mathbf{v}_0^K, \mathbf{v}_1^K, \cdots, \mathbf{v}_{M-1}^K]^T \tag{15.86}$$

FINT 模型将所有 Field 的最终表示进行拼接：

$$\mathbf{h}_0 = \mathbf{v}_0^K \| \mathbf{v}_1^K \| \cdots \| \mathbf{v}_{M-1}^K \| \tag{15.87}$$

并输入多层 DNN：

$$\mathbf{h}_{l+1} = \mathrm{ReLU}\left(W_l \mathbf{h}_l + \mathbf{b}_l\right) \tag{15.88}$$

Field-aware Interaction Layer 主要通过显式的 Hadamard 乘积构造 Field 之间的组合关系，而 DNN 则进一步在语义空间中学习更加复杂的非线性高阶特征交互。因此，两者形成互补：前者负责保留 Field 结构并构造显式交互，后者负责挖掘更加灵活的隐式交互。最终，DNN 输出经过 Sigmoid 函数得到用户对目标 Item 的点击概率：

$$\hat{y} = \sigma\left(\mathrm{FFN}(\mathbf{h}_0)\right) \tag{15.89}$$

对于 CTR 二分类任务，FINT 模型采用 Binary Cross Entropy 作为训练目标。

总体来看，FINT 模型的核心思想是“Field-aware 的高阶交互”，即通过为每个 Field 持续维护独立的交互向量，使高阶特征交互过程中不会丢失 Field 边界；同时结合 DNN 进一步建模非线性特征关系。与依赖 Self-Attention 的 AutoInt 相比，FINT 直接采用 Hadamard 乘积和 Field-wise 加权聚合实现特征交互，在保持 Field 结构的同时具有较为简洁的计算形式。

15.5.9 GDCN

GDCN 模型源自于论文《Towards Deeper, Lighter and Interpretable Cross Network for CTR Prediction》[**wang2023towards**]。其核心目标是在保持显式特征交互能力的同时，进一步挖掘高阶交叉特征，并降低无效高阶交互带来的噪声。GDCN 模型在 DCN-V2 的 Cross Network 基础上引入信息门控（Information Gate），通过门控机制对不同阶次的交叉特征进行自适应筛选，使模型能够更加关注当前样本中具有预测价值的特征交互。

GDCN 模型的网络结构如图15.9所示。GDCN 模型整体由 Embedding Layer、Gated Cross Network（GCN）和

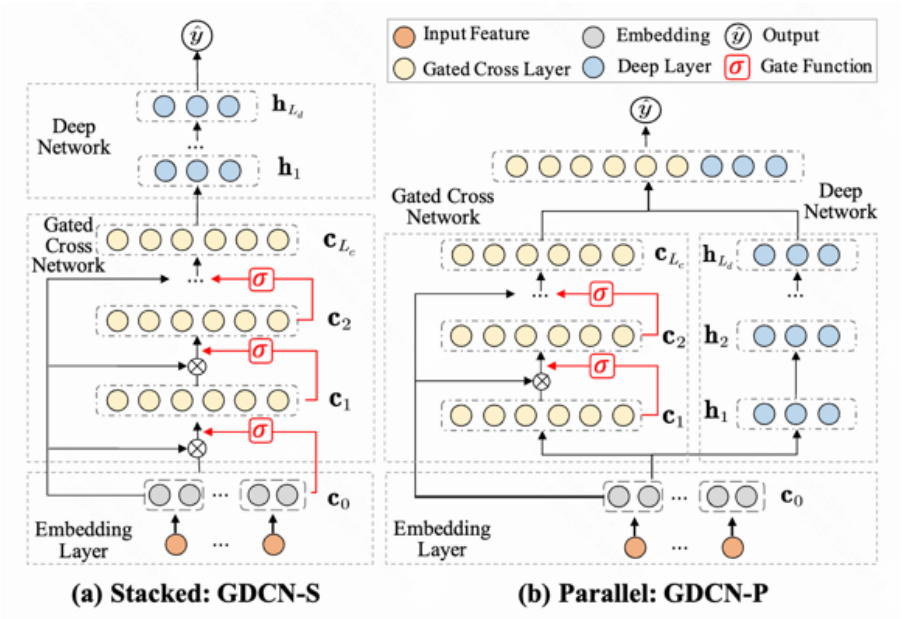


图 15.9: GDCN 模型的网络结构。

Deep Neural Network（DNN）三部分组成。首先，Embedding Layer 将高维稀疏特征转换为低维稠密表示，并将不同 Field 的 Embedding 进行拼接得到基础特征表示 $\mathbf{c}_0$。随后，GCN 模块通过显式的 Cross Layer 逐层构造高阶特征交互，并利用 Information Gate 对新产生的高阶交叉信息进行动态加权；与此同时，DNN 负责学习隐式的高阶非线性特征交互。根据 GCN 与 DNN 的连接方式不同，GDCN 模型进一步提供了串联结构 GDCN-S 和并联结构 GDCN-P 两种不同的形式。

需要注意的是，GCN 是 GDCN 的核心模块，用于显式建模不同阶次的特征交互。对于第 $l+1$ 个 Gated Cross Layer，其计算过程可以表示为：

$$\mathbf{c}_{l+1} = \underbrace{\mathbf{c}_0 \odot \left(\mathbf{W}_l^{(c)}\mathbf{c}_l + \mathbf{b}_l\right)}_{\text{Feature Crossing}} \odot \underbrace{\sigma\left(\mathbf{W}_l^{(g)}\mathbf{c}_l\right)}_{\text{Information Gate}} + \mathbf{c}_l \tag{15.90}$$

其中，$\mathbf{c}_0$ 为 Embedding Layer 输出的一阶特征表示，$\mathbf{c}_l$ 为第 l 层 Cross Layer 的输出，$\mathbf{W}_l^{(c)}$ 和 $\mathbf{W}_l^{(g)}$ 分别表示 Cross Matrix 和 Gate Matrix，$\mathbf{b}_l$ 为偏置项。该过程包含两个关键步骤。首先，**Feature Crossing** 利用 $\mathbf{c}_0$ 与当前高阶表示 $\mathbf{c}_l$ 进行逐元素交互，并通过 $\mathbf{W}_l^{(c)}$ 学习不同特征之间的交叉关系，从而逐层生成更高阶的显式交互特征。其次，**Information Gate** 通过 $\mathbf{W}_l^{(g)}\mathbf{c}_l$ 经过 Sigmoid 函数得到逐维的 Gate 值，并与新产生的交叉特征进行逐元素相乘，从而自适应地增强重要交叉特征、抑制无效交叉特征。

随着 Cross Layer 不断加深，高阶特征组合数量会快速增长，其中部分交互可能包含大量噪声。GDCN 在每一层引入 Information Gate，使不同阶次的交叉信息能够根据当前输入样本进行动态筛选，从而在利用深层高阶交互的同时控制噪声累积，并赋予模型一定的实例级可解释性。最后，通过残差连接保留上一层的交互信息，使最终表示同时包含从低阶到高阶的多阶特征交互。

除了显式的 GCN 模块之外，GDCN 模型还引入了 DNN 模块来对隐式特征交互进行建模。DNN 的基本计算过程为：

$$\mathbf{h}_{l+1} = f\left(\mathbf{W}_l\mathbf{h}_l + \mathbf{b}_l\right) \tag{15.91}$$

其中，$f(\cdot)$ 通常采用 ReLU 激活函数。通过多层非线性变换，DNN 能够学习 Cross Network 难以显式表示的复杂高阶特征关系。根据 GCN 与 DNN 的组合方式，GDCN 包含两种网络结构：

- **GDCN-S（Stacked）**：首先将 Embedding 输出 $\mathbf{c}_0$ 输入 GCN 进行多层显式交互，得到 $\mathbf{c}_{L_c}$，再将其输入 DNN 进行进一步的隐式特征交互，最终得到 $\mathbf{c}_{final} = \mathbf{h}_{L_d}$。
- **GDCN-P（Parallel）**：将 $\mathbf{c}_0$ 同时输入 GCN 和 DNN，两路分别学习显式与隐式特征交互，最后将二者输出

进行拼接，即 $\mathbf{c}_{final} = [\mathbf{c}_{L_c} \| \mathbf{h}_{L_d}]$。

最终，GDCN 模型将 $\mathbf{c}_{final}$ 输入输出层得到点击概率：

$$\hat{y} = \sigma\left(\mathbf{w}_{logit}^{\top} \mathbf{c}_{final}\right) \tag{15.92}$$

并采用 CTR 任务中常用的 Binary Cross-Entropy（Log Loss）进行模型训练。

GDCN 模型可以看作是 DCN-V2 的进一步扩展。当去除 Information Gate，或者将所有 Gate 值设置为 1 时，GDCN 模型就会退化为 DCN-V2 模型。二者的主要区别在于，DCN-V2 模型对当前层产生的交叉特征进行统一聚合，而 GDCN 模型在每一层进一步引入逐维的 Information Gate，根据输入实例自适应地调整不同交叉特征的重要性。因此，GDCN 模型能够在更深的 Cross Network 中有效过滤无效高阶交互，提升模型对深层交叉信息的利用能力。需要注意的是，GDCN 模型中的 Gate 与 DCN-V2 模型中的 Gate 具有不同的作用。DCN-V2 模型借鉴 MMoE 的思想，通过多个 Expert 对 Cross Matrix 进行子空间分解，并利用 Gate 组合不同 Expert，主要目的是降低 Cross Matrix 的非 Embedding 参数量；而 GDCN 模型的 Information Gate 则直接作用于交叉特征，用于动态筛选具有预测价值的交互信息。

此外，GDCN 模型进一步提出了 Field-level Dimension Optimization（FDO），通过对训练得到的 Field Embedding 进行 PCA 分析，根据不同 Field 的信息保留比例自适应确定 Embedding 维度。相比所有 Field 采用统一维度的方式，FDO 能够压缩不重要 Field 的 Embedding 参数，同时降低 GCN 模块中的 Cross Matrix 和 Gate Matrix 规模，从而进一步提升模型的部署效率。因此，GDCN 模型在**高阶特征交互、动态特征筛选和模型轻量化**之间取得了较好的平衡。

15.5.10 IPA

IPA 模型源自于腾讯在 2024 年的论文《Towards Unifying Feature Interaction Models for Click-Through Rate Prediction》[**kang2025towards**]。与 FM、DCN-V2、xDeepFM 等针对特定特征交互方式设计的模型不同，IPA 模型并不是一个单独的 CTR 预测模型，而是提出了一种**模块化的特征交互统一框架**，用于从统一视角分析和构建不同的特征交互模型。

IPA 模型将 CTR 模型中的特征交互过程拆解为三个相互独立的核心组件：交互函数（Interaction Function）、层池化（Layer Pooling）和层聚合（Layer Aggregator）。其中，Interaction Function 负责定义两个 Embedding 之间如何进行显式交互；Layer Pooling 负责按照一定的规则逐层构造更高阶的特征交互；Layer Aggregator 则负责将不同阶次的交互信息进行聚合，形成最终的特征交互表示。通过对三个组件分别进行组合，可以统一描述大量已有的特征交互模型，并进一步探索新的模型结构。

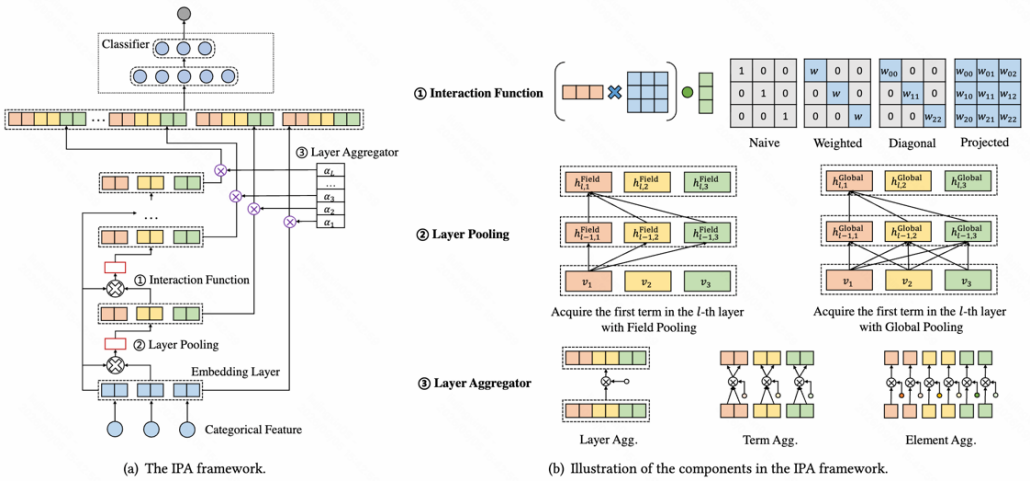


图 15.10: IPA 模型的网络结构。

IPA 模型的网络结构如图15.10所示。对于输入特征，首先通过 Embedding 层将原始的稀疏特征转换为稠密 Embedding，并将各 Field 的 Embedding 作为第一阶特征表示。随后，IPA 模型通过 Interaction Function 对不同

Embedding 进行显式交互。给定两个 Embedding $\mathbf{t}_i, \mathbf{t}_j \in \mathbb{R}^K$，其交互结果可以统一表示为：

$$\mathbf{t}_{i,j} = f(\mathbf{t}_i, \mathbf{t}_j) \in \mathbb{R}^K \tag{15.93}$$

IPA 模型进一步将现有模型中的 Interaction Function 归纳为四种典型形式：**Naive**、**Weighted**、**Diagonal** 和 **Projected**。它们分别对应不同程度的参数化特征交互方式，例如 FM 采用 Naive 形式，FwFM 和 xDeepFM 采用 Weighted 形式，FvFM 采用 Diagonal 形式，而 FmFM 和 DCN-V2 则采用更加灵活的 Projected 形式。因此，Interaction Function 主要决定了两个特征如何进行基本交互。

在得到基础交互操作后，IPA 模型通过 Layer Pooling 逐层构造高阶特征交互。由于所有特征的组合数量会随着交互阶数快速增长，实际模型通常不会显式保留所有组合，而是按照特定规则将上一阶交互结果与原始 Embedding 进行组合和聚合。IPA 模型将 Layer Pooling 进一步划分为 **Field-wise Layer Pooling** 和 **Global-wise Layer Pooling** 两种形式。前者按照 Field 分别聚合交互信息，使每个 Field 保持独立的交互表示；后者则对不同 Field 产生的交互信息进行全局聚合。DCN-V2 模型采用 Field Pooling 构建高阶 Cross Layer，而 xDeepFM 模型的 CIN 则采用 Global Pooling。因此，Layer Pooling 主要决定了高阶交互如何逐层生成和组织。

经过多层交互后，不同阶次的特征交互信息还需要进一步融合。IPA 模型通过 Layer Aggregator 将不同交互层中的信息聚合为最终的特征交互表示。根据聚合粒度的不同，可以采用 Direct Aggregation、Layer Aggregation、Term Aggregation 和 Element Aggregation 等方式，分别在层级、Term 级甚至 Embedding 元素级对不同交互信息进行加权。其中，Direct Aggregation 不引入额外权重，而 Layer Aggregation 则为不同阶次的交互层学习独立权重。四种 Layer Aggregator 的具体示例如下表15.1所示，其中 L 表示交互层数，M 表示每层的 Term 数量，K 表示 Embedding 维度，$\|$ 表示拼接操作。因此，Layer Aggregator 主要决定了不同阶次交互信息如何被综合利用。

表 15.1: IPA 模型中 Layer Aggregator 的具体示例。

聚合方式	权重	参数量	公式
Direct Aggregation	1.0	0	$\mathbf{r} = \sum_{l=1}^{L} \Vert_{n=1}^{M} \{\mathbf{t}_{l,n}\}$
Layer Aggregation	α_l	L	$\mathbf{r} = \sum_{l=1}^{L} \alpha_l \Vert_{n=1}^{M} \{\mathbf{t}_{l,n}\}$
Term Aggregation	$\alpha_{l,n}$	LM	$\mathbf{r} = \sum_{l=1}^{L} \Vert_{n=1}^{M} \{\alpha_{l,n} \cdot \mathbf{t}_{l,n}\}$
Element Aggregation	$\alpha_{l,n,k}$	LMK	$\mathbf{r} = \sum_{l=1}^{L} \Vert_{n=1}^{M} \{\Vert_{k=1}^{K} \{\alpha_{l,n,k} \cdot \mathbf{t}_{l,n,k}\}\}$

通过将 Interaction Function、Layer Pooling 和 Layer Aggregator 进行模块化拆分，IPA 能够从统一框架下解释不同 CTR 特征交互模型的设计差异。例如，DCN-V2 模型可以表示为 Projected Product、Field Pooling 和 Direct Aggregation 的组合，而 xDeepFM 模型则可以对应另一组 Interaction Function 和 Layer Pooling 配置。更重要的是，该框架不仅能够分析已有模型，还可以通过组合不同组件构建新的特征交互模型。例如，论文基于 Projected Product、Field Pooling 和 Layer Aggregation 构建了新的 PFL 模型，从而验证了不同组件组合所带来的模型设计空间。

总的来说，IPA 模型的核心贡献并不是提出一种新的特征交互算子，而是将现有 CTR 模型中的特征交互过程抽象为**“如何交互、如何构造高阶交互、如何聚合不同阶次信息”**三个相互独立的模块，为理解和设计 CTR 特征交互模型提供了统一的分析框架。

15.5.11 特征交互建模讨论

对于工业推荐模型而言，特征交互一直是模型建模能力的重要组成部分。推荐模型通常同时包含大量 User、Item、Context 以及 Behavior 特征，不同特征之间往往存在复杂的组合关系。例如，用户画像与物品类别之间、用户历史行为与当前物品之间、Context 与物品之间都可能形成具有实际业务意义的交互。如果对所有特征进行完全交互，不仅会带来较高的计算和参数开销，其中大量交互实际上也缺乏明确的业务意义。因此，如何在有限的计算预算下，从大量潜在的特征组合中学习有效的交互关系，一直是工业推荐模型中的重要研究问题。

围绕这一问题，近年来的特征交互模型经历了一条较为清晰的演进路线。从早期的 DCN、DCN-V2、FiBiNET，到基于 Attention 机制的 AutoInt，再到 AFN、FED、CAN、FINT、GDCN 以及 IPA，可以看到特征交互建模逐渐从**固定的显式交叉**向**动态的、细粒度的、结构化的交互学习**演进。不同模型虽然采用了不同的网络结构，但其核心目标基本一致，即希望模型能够更加有效地回答三个问题：

- 哪些 Feature 应该进行交互？
- 不同 Feature 应该以什么方式进行交互？
- 不同交互关系的重要程度应该如何衡量？

DCN 模型是这一方向中的代表性工作之一。传统 DNN 模型虽然能够通过多层非线性变换隐式学习特征之间的关系，但模型需要依赖较深的网络结构才能逐步发现高阶组合关系，而且这种隐式交互缺乏明确的结构约束。DCN 模型通过 Cross Network 显式构造特征交叉，使模型能够以较低的计算成本学习高阶特征交互，从而在保持较低复杂度的同时增强模型对于组合关系的表达能力。DCN-V2 模型进一步将 Cross Network 中的交互过程参数化，引入更加灵活的交叉变换，使模型不再局限于 DCN 中相对受限的交互形式，从而显著提升了 Cross Network 的表达能力。

然而，显式 Cross Network 仍然存在一个重要问题：交互关系在很大程度上由预先定义的网络结构决定，而真实推荐场景中的有效特征组合往往具有较强的动态性和异质性。在不同用户、不同物品以及不同上下文中，真正有价值的交互关系可能并不相同。因此，后续研究开始尝试让模型自适应地学习特征之间的交互关系。AutoInt 模型通过 Self-Attention 机制对不同特征域进行建模，使每个特征能够根据其他特征的信息动态计算 Attention 权重，从而自动学习不同特征域之间的依赖关系。与固定的 Cross Network 相比，这种方式不再需要预先指定具体的交叉组合，而是通过数据驱动的方式学习更加灵活的特征交互。

在此基础上，FiBiNET 模型进一步关注不同特征重要程度以及不同 Embedding 维度之间的交互差异，通过 Squeeze-and-Excitation 等机制对特征进行重新加权，并结合双线性交互增强不同特征之间的组合表达能力。这类方法说明，特征交互并不只是简单地计算两个 Embedding 之间的乘积，更重要的是需要考虑不同特征的重要程度以及不同交互关系的表达方式。

随着模型进一步发展，研究者开始探索更加灵活的高阶交互建模方式。AFN 模型通过对数变换显式建模不同阶数的特征交互，使模型能够在较低网络深度下学习高阶特征组合；FED 模型则进一步从特征交互的结构化建模角度出发，对不同阶数和不同组合关系进行更加细粒度的建模。这一阶段的研究逐渐体现出一个重要趋势：特征交互的目标已不再只是简单地“构造高阶特征交叉”，而是在可控计算成本下选择更加有效的交互方式。

随后，CAN、FINT 等模型进一步将研究重点推进到更加细粒度的特征域的交互。与传统 Cross Network 直接对整体输入向量进行统一交叉不同，这类方法更加关注不同特征域之间的组合关系，通过独立参数、动态权重或者结构化交互模块，对不同特征域之间的交互进行更加精细的建模。尤其是在工业推荐场景中，User、Item、Author、Category、Tag 以及行为序列等特征具有明确的语义，不同特征域之间的交互模式通常存在显著差异。因此，相比起简单地将所有 Embedding 进行 Concat 之后交给 DNN，显式的特征域级别的交互能够更加直接地利用这些结构信息，并减少模型完全依赖深层非线性网络自行发现交互关系的负担。

GDCN 模型则进一步探索了更加灵活的显式交叉结构，通过对特征交叉过程进行改进，使模型能够在较低计算成本下获得更强的高阶交互表达能力。与此同时，IPA 等方法开始进一步从交互函数本身的角度重新审视特征交互，将不同 Embedding 之间的交互过程从简单的固定运算扩展为更加通用的 Interaction Function，并通过 Layer Pooling 和 Layer Aggregation 等机制对不同层次的交互结果进行融合。这类方法体现出特征交互研究的一个重要变化：研究重点逐渐从“如何设计一种更强的交叉结构”，转向“如何定义更加灵活的 Interaction Function，以及如何有效组织和聚合不同层次的交互信息”。

因此，从 DCN 到 DCN-V2，再到 AutoInt、FiBiNET、AFN、FED、CAN、FINT、GDCN 以及 IPA，可以将这一系列工作的演进概括为如图15.11所示的发展脉络。其背后的核心变化，是 Feature Interaction 逐渐从一种人为设计的固定运算，演进为一种由模型根据数据动态学习的表示变换。换句话说，研究者一直在尝试减少对于人工特征工程的依赖，让模型能够在大量潜在特征组合中自动发现真正有价值的交互关系。从工业推荐系统的角度来看，这种演进也可以进一步抽象为：

$$\text{Feature Fields} \rightarrow \text{Interaction Learning} \rightarrow \text{Feature Representation} \tag{15.94}$$



图 15.11: 特征交互模型的发展脉络。

这里的交互学习（Interaction Learning）并不仅仅意味着增加交叉阶数，而是需要同时考虑交互的选择、形式和权重。这也正是 Attention、Gating 以及各种动态 Interaction 结构不断进入推荐模型的重要原因。

不过，值得注意的是，随着推荐模型的发展，尤其是在 GDCN、IPA 等工作之后，专门针对特征交互设计新型交叉结构的研究热度开始逐渐下降。这并不意味着特征交互的重要性降低了，而是因为**特征交互逐渐不再被视为一个独立且难以解决的模型瓶颈**。一方面，现代推荐模型的整体表达能力已经显著增强。随着 Embedding 维度、模型参数规模以及训练数据规模不断增加，传统的二阶或者固定阶数特征交叉所带来的收益逐渐受到限制。另一方面，随着 GPU 等计算资源的持续发展，工业推荐系统能够承受更加复杂的模型结构，使得过去为了降低计算成本而专门设计的轻量级交互模块，其必要性也在逐渐下降。更重要的是，Attention、Transformer 以及 Mixture-of-Experts 等通用模型结构本身就具备较强的特征关系建模能力。此时，与其不断设计更加精巧的特征交叉公式，不如直接提升模型的整体表示能力，让模型通过更通用的架构自动学习复杂的特征交互。

这一变化可以理解为推荐模型研究范式的一次转变：过去的核心问题是“**如何设计更好的特征交互模块**”，而现在越来越多的问题转变为“**如何构建具有更强 Scaling 能力的通用推荐模型**”。在传统模型中，如果希望捕捉 User、Item、Context 和 Behavior 之间的复杂关系，通常需要显式设计 Cross Network、Bilinear Interaction、Attention 或者其他专门的交互模块；而在 Transformer 架构下，可以将不同特征表示为统一的 Token 序列，通过 Self-Attention 让不同 Token 之间直接建立信息交互。此时，原本需要专门设计的特征交互，很大程度上已经成为 Transformer Layer 本身的一种基础能力。

因此，随着推荐模型逐渐向更大规模、更高维度以及更加统一的架构发展，特征开始从一个相对独立的研究问题逐渐融入模型主体。尤其是以 RankMixer[**zhu2025rankmixer**] 为代表的新一代推荐大模型开始探索将传统特征工程进一步 Token 化，把 User、Item、Context、Behavior 等不同来源的特征映射到统一的 Token 空间，再通过 Token Mixer、Self-Attention、Transformer 以及 Mixture-of-Experts 等通用结构进行统一建模。在这种范式下，模型不再需要针对每一种特征组合单独设计 Cross 结构，而是通过统一的 Token-level Representation Learning 自动学习不同特征之间的复杂依赖关系。

从这个角度来看，特征交互并没有真正消失，而是发生了“**模块化设计向通用架构内生能力的迁移**”。过去，特征交互是模型需要显式解决的问题；而在 Transformer-based Recommendation Model 中，特征交互逐渐成为模型通过 Attention 机制自然获得的基础能力。随着计算资源和训练数据进一步扩大，模型 Scaling 带来的收益也越来越明显，专门设计复杂特征交叉结构的边际收益相对下降，而提升模型规模、Token 表示能力以及统一建模能力逐渐成为更加重要的研究方向。

因此，GDCN、IPA 之后，特征交互研究并非简单地“停止发展”，而是逐渐从独立的特征交互研究融入统一的表示学习与大规模推荐模型框架之中。对于当前的推荐模型而言，当计算预算有限时，如何通过精巧的交互模块以较低成本捕捉有效交互仍然具有重要价值；但当模型进入更大规模、算力更加充足的场景后，Transformer Layer 等通用架构已经能够以更加统一的方式捕捉复杂的跨特征依赖关系，此时继续针对特征交叉进行高度定制化的结构设计，其收益与必要性都会逐渐降低。

这一趋势也意味着，推荐模型正在从过去的“**Feature Engineering + Interaction Engineering**”范式逐渐走向“**Tokenization + General Representation Learning + Scaling**”的新范式。如何将传统推荐系统中的大量异构特征统一表示为 Token，并利用更加通用的 Transformer、Token Mixer 以及 Mixture-of-Experts 架构进行高效建模，正在成为下一阶段推荐模型发展的重要方向。以 RankMixer 为代表的推荐大模型正是在这一背景下逐渐兴起。关于 Token 化推荐、Transformer-based Recommendation 以及推荐大模型的 Scaling，我们将在后续第36章中进行详细讨论。

15.6 特征筛选建模

随着推荐模型中的 Feature 数量不断增加，特征工程所面临的问题也逐渐从“如何构造更多有效特征”转变为“如何从大量特征中筛选真正有价值的特征”。在工业推荐系统中，User、Item、Context、Behavior 等 Feature 通常具有不同的数据来源、更新频率、计算成本和业务价值。随着特征数量不断增长，简单地将所有特征输入模型并不一定能够带来更好的效果，反而可能引入大量冗余、噪声甚至存在数据泄漏风险的特征，同时增加训练和在线推理的计算成本。

因此，特征筛选并不仅仅是传统意义上的特征选择（Feature Selection）问题，而是一个综合考虑特征重要性、特征冗余、特征成本、特征质量以及数据稳定性的问题。一个真正适合工业推荐系统的特征选择策略，需要同时回答以下几个问题：

- **特征重要性（Feature Importance）**：哪些特征对于当前预测任务真正有贡献？
- **特征冗余（Feature Redundancy）**：不同特征之间是否包含大量重复的信息？
- **特征成本（Feature Cost）**：特征的生成、存储、传输和在线计算成本是否值得？
- **特征质量（Feature Quality）**：特征是否具有足够的覆盖率、准确性和稳定性？
- **数据漂移（Data Drift）**：特征的数据分布是否随着时间发生变化？
- **特征泄漏（Feature Leakage）**：特征是否错误地使用了预测时刻之后才能获得的信息？
- **特征治理（Feature Governance）**：如何在整个生命周期内对特征进行统一管理、监控和下线？

因此，工业场景中的特征筛选实际上更接近一个多目标优化问题。可以将一个特征 f 的综合价值抽象表示为：

$$V(f) = \lambda_1 I(f) - \lambda_2 R(f) - \lambda_3 C(f) + \lambda_4 Q(f) - \lambda_5 D(f) \tag{15.95}$$

其中，$I(f)$ 表示特征重要性（Feature Importance），$R(f)$ 表示特征冗余（Feature Redundancy），$C(f)$ 表示特征成本（Feature Cost），$Q(f)$ 表示特征质量（Feature Quality），而 $D(f)$ 可以用于衡量特征的数据漂移程度（Data Drift）。不同业务场景可以根据模型效果、计算预算和稳定性要求对这些因素设置不同的权重。需要注意的是，这一表达式更多是一种概念上的统一描述，在实际工业系统中通常不会直接采用如此简单的线性评分，而是结合离线实验、在线 A/B 实验以及特征监控进行综合决策。

15.6.1 特征重要性

特征选择 (Feature Selection) 首先需要解决的是特征重要性问题，即判断一个特征对于最终预测结果到底有多大贡献。传统机器学习中可以通过线性模型的权重、树模型的特征增益 (Feature Gain)、置换重要性 (Permutation Importance) 等方式衡量特征重要性，这里不再赘述这些过于久远的方法；而在深度推荐模型中，由于特征通常首先经过 Embedding 层，再经过特征交互 (Feature Interaction) 层和多层非线性变换层，因此很难再通过某一个显式参数直接解释单个特征的重要程度。

在这一背景下，一个重要的发展方向是让模型**自动学习不同信息单元的贡献程度**，而不是依赖人工规则直接决定每个特征是否保留。值得注意的是，这一思想并非最早产生于推荐系统。Swietojanski 等人提出的 Learning Hidden Unit Contributions（LHUC）[**swietojanski2016learning**] 便是这一思想在语音识别领域中的早期代表性工作。LHUC 最初用于无监督声学模型自适应（Unsupervised Acoustic Model Adaptation），其核心目标并不是进行特征选择，而是针对不同说话人或者声学环境，对预训练 DNN 中的隐藏单元进行自适应调整。其基本思想非常

简单：在不改变基础网络参数的情况下，为不同隐藏单元引入额外的幅度参数，根据当前说话人动态调整不同隐藏单元的贡献。

具体来说，假设第 l 层的隐层表示为：

$$\mathbf{h}^{(l)} = [h_1^{(l)}, h_2^{(l)}, \cdots, h_K^{(l)}] \tag{15.96}$$

LHUC 模型为每一个隐藏单元引入一个与说话人相关的参数 $r_j^{(l)}$，并通过非线性函数将其转换为贡献权重（Contribution Weight），即：

$$\tilde{h}_j^{(l)} = \xi(r_j^{(l)}) h_j^{(l)} \tag{15.97}$$

其中 $\xi(\cdot)$ 用于将参数映射到合适的范围。这样，模型就可以在保持原始 DNN 主体参数基本不变的情况下，对不同隐藏单元进行增强或者抑制。其过程可以抽象为：

$$\mathbf{h} \rightarrow \underbrace{\mathbf{g}}_{\text{Unit Contribution}} \odot \mathbf{h} \rightarrow \tilde{\mathbf{h}} \tag{15.98}$$

LHUC 的重要意义在于，它将“不同信息单元应该贡献多少”从一个固定的网络参数问题转变为一个可以根据当前输入条件动态调整的问题。虽然 LHUC 并不是为推荐系统中的特征选择设计的，但其贡献度建模（Contribution Modeling）的思想与后来推荐模型中的 Feature Re-weighting、Attention 和 Gating 具有较强的关联性。

随后，这一思想逐渐进入推荐模型的特征重要性建模。例如，FiBiNET 模型在其特征交互框架中引入 Squeeze-and-Excitation（SE）机制，对不同特征域进行动态重标定 [**huang2019fibinet**]。SE-Net 的核心思想是通过显式建模不同通道之间的依赖关系，学习每个通道的响应强度，并据此对原始表示进行重新加权 [**hu2018squeeze**]。FiBiNET 模型进一步将这一思想应用到推荐系统中的特征域，使模型能够学习不同特征域的重要程度。对于输入特征的 Embedding $\mathbf{e}_i$，可以将这一类方法抽象表示为：

$$g_i = f(\mathbf{e}_1, \mathbf{e}_2, \cdots, \mathbf{e}_N) \tag{15.99}$$

其中 g_i 表示第 i 个特征的重要性权重，然后通过：

$$\tilde{\mathbf{e}}_i = g_i \mathbf{e}_i \tag{15.100}$$

对特征表示进行动态重加权。当 g_i 较大时，说明当前特征具有较高的贡献；当 g_i 较小时，则相当于抑制该特征的信息。

与传统的硬性特征选择（Hard Feature Selection）相比，这种方式并不直接回答“特征是否保留”，而是回答“特征在当前输入条件下应该贡献多少”。因此，可以将其视为一种**软性特征选择（Soft Feature Selection）**。尤其是在推荐系统中，同一个特征对于不同用户、物品以及上下文可能具有完全不同的重要程度。例如，用户的历史行为特征可能对于活跃用户非常重要，而对于新用户则可能由于数据稀疏而贡献有限。因此，动态特征权重相比静态特征排序更加符合推荐系统高度个性化的特点。

在此基础上，Attention、Gating 以及其他动态交互结构进一步将特征重要性从 Field-Level Weighting 扩展到更加细粒度的 Token-Level 或者 Dimension-Level Weighting，使模型能够根据当前输入动态决定不同信息单元之间的信息流动。此时，特征重要性与特征交互之间的边界也开始逐渐模糊：一个特征的重要程度不仅取决于特征自身，还取决于它与其他特征之间的交互关系。

因此，从 LHUC 到 SE-Block，再到 Attention 和 Gating，可以看到一个值得关注的演进趋势：

$$\text{Static Importance} \rightarrow \text{Adaptive Contribution} \rightarrow \text{Feature Re-weighting} \rightarrow \text{Fine-grained Dynamic Selection} \tag{15.101}$$

这一演进说明，现代推荐模型中的特征选择已经不再局限于传统意义上的“删除无用特征”。在模型表达能力不断增强的情况下，更常见的做法是让模型保留较大的特征空间，同时通过 Attention、Gating 或者 Re-weighting 机制动态控制不同特征的贡献。这样既可以避免过早进行 Hard Selection 造成的信息损失，又能够让模型根据 User、Item 和 Context 的信息动态利用不同的特征从而实现个性化。

从更广义的角度来看，LHUC 所体现的 Contribution Modeling 思想也揭示了特征工程与模型架构逐渐融合的趋势。早期特征工程通常由人工决定“使用哪些特征”，而现代模型则越来越倾向于由网络自身学习“当前场景下应该使用哪些特征以及使用多少”。因此，特征重要性正在逐渐从一个独立的特征工程问题转变为模型内部表示

学习和信息路由的一部分。

15.6.2 特征冗余

除了特征重要性之外，大规模推荐系统中的另一个重要问题是特征冗余。随着特征工程不断发展，不同特征之间往往会出现较强的相关性甚至信息重叠。例如，用户的历史点击次数、近 7 天点击次数、近 30 天点击次数以及用户活跃等级之间可能包含大量相似信息；Item 的 Category、Sub-Category、Tag 以及人工规则生成的多个统计特征也可能存在较强相关性。

特征冗余并不意味着相关特征一定应该删除。在深度模型中，多个具有相关性的特征有时能够从不同角度提供互补信息，尤其是在数据稀疏、特征缺失或者不同特征具有不同更新频率的情况下。因此，工业特征筛选更关注的是**边际信息增益**，即在已有特征集合的基础上，新增加一个特征是否还能带来稳定的收益。可以将这一过程理解为：

$$\Delta M = M(\mathcal{F} \cup \{f\}) - M(\mathcal{F}), \tag{15.102}$$

其中 $\mathcal{F}$ 表示当前已经使用的特征集合，$M(\cdot)$ 表示模型效果。当 ΔM 长期接近于零时，即使某个特征本身具有一定的预测能力，将其继续加入模型的实际价值也可能非常有限。

因此，在工业实践中，特征选择往往不能仅仅根据单特征的重要性进行排序，而需要结合特征之间的相关性以及加入特征后的增量收益进行综合判断。尤其是在推荐模型的长期迭代过程中，一个非常常见的现象是：**模型中的特征只增不减**。每一次模型迭代都可能加入新的用户统计特征、物品统计特征、行为序列特征或者场景特征，单次增加的特征数量可能并不多，但经过数十次甚至上百次迭代之后，模型输入特征的规模可能已经变得非常庞大。

特征规模持续增长不仅会增加模型参数量，还会进一步带来 Embedding 参数规模增长、训练显存增加、模型训练时间变长以及在线推理成本上升等问题。更重要的是，新增特征带来的模型收益通常会随着迭代逐渐降低。当模型已经积累了大量特征之后，继续加入新的特征可能只能获得非常有限的离线指标提升，甚至出现模型参数量不断增长但核心业务指标基本不再提升的情况。此时，算法工程师往往需要从“继续增加特征”转向“重新审视已有特征”。

因此，在实际的模型长期迭代过程中，**特征精简（Feature Pruning）**往往并不是模型开发初期的重点，而更多发生在模型已经变得较为臃肿、收益逐渐饱和或者训练资源开始成为瓶颈之后。一个典型的特征精简过程可以表示为：

$$\text{特征累积} \rightarrow \text{模型饱和} \rightarrow \text{特征精简} \rightarrow \text{消融验证} \tag{15.103}$$

其中，特征累积表示随着模型不断迭代，特征数量持续增加；模型饱和表示新增特征带来的收益逐渐降低，模型整体进入收益瓶颈；特征精简则尝试从已有特征中删除贡献较低或者高度冗余的特征；最后通过消融验证验证精简后的模型是否能够在减少特征和模型规模的同时保持原有的效果。

在具体实践中，通常不建议一次性删除大量特征，因为不同特征之间可能存在复杂的协同关系，单独看贡献较低的特征组合在一起后可能仍然具有一定价值。因此，更常见的做法是按照特征来源、业务含义或者相关性将特征划分为若干组，然后**分批次进行特征消融**。例如，可以分别对用户画像特征、用户行为特征、物品特征、场景特征等进行消融，并观察模型离线指标和线上业务指标的变化。对于一个待删除的特征组 $\mathcal{G}$，可以定义其消融带来的性能变化为：

$$\Delta M_{\mathcal{G}} = M(\mathcal{F}) - M(\mathcal{F} \setminus \mathcal{G}) \tag{15.104}$$

如果删除 $\mathcal{G}$ 之后模型效果基本不变，即：

$$\Delta M_{\mathcal{G}} \approx 0 \tag{15.105}$$

那么说明这一组特征在当前特征集合中可能已经缺乏明显的边际价值，可以进一步考虑删除。反之，如果删除之后核心指标出现明显下降，则说明这些特征仍然提供了有效信息，需要继续保留。

需要特别注意的是，**离线消融结果并不能完全替代线上验证**。部分特征可能对离线 AUC、Log Loss 等指标影响较小，但会通过候选排序、长尾 Item 曝光或者特定用户群体产生线上业务影响。因此，在工业推荐系统中，特征精简通常需要经过“离线消融 → 小流量实验 → 全量验证”的过程，在确认模型效果基本不受影响后再正式下线特征。

从更长周期来看，特征冗余的治理本质上是在解决推荐模型中的**特征负债**问题。类似于软件工程中的技术负债，特征一旦被加入模型，后续往往会因为依赖关系、历史实验或者业务方使用而很难主动删除，最终形成大量“没人敢删、但又说不清价值”的历史特征。如果长期缺乏治理，特征数量就会不断累积，使模型越来越复杂，同时也增加后续问题定位和模型迭代的成本。

因此，优秀的特征工程并不是不断向模型中加入更多特征，而是在持续获得有效信息的同时，及时识别和删除低价值、重复或者高成本的特征，使模型始终维持在一个合理的复杂度范围内。对于工业推荐系统而言，真正值得追求的并不是特征越多越好，而是在有限的计算和工程成本下，尽可能获得更多的有效信息。这也意味着，特征选择与特征工程并不是两个相互独立的过程，而应该贯穿推荐模型的整个生命周期：模型早期通过特征工程不断探索新的有效信息，模型进入成熟阶段后则通过特征选择和特征精简持续清理冗余特征，在模型效果、计算成本和系统复杂度之间取得平衡。

15.6.3 特征成本

在工业推荐系统中，一个特征是否值得保留，除了考虑其对模型效果的贡献，还需要综合评估其计算、存储、网络和在线服务成本。传统离线数据集上的特征选择往往主要关注模型精度，但对于线上推荐系统而言，一个特征可能需要经过离线计算、实时计算、特征存储查询、跨服务 RPC 调用或者复杂的聚合操作才能获得。因此，特征成本可以进一步划分为：

$$C(f) = C(f) + C(f) + C(f) + C(f) \tag{15.106}$$

分别表示特征的计算、存储、网络传输以及在线延迟成本。特别是在推荐系统的粗排和精排阶段，特征成本往往会直接影响模型的在线推理效率。例如，一个只能带来极小 AUC 提升、却需要增加大量在线 RPC 调用或者复杂实时计算的特征，在工业系统中可能并不值得使用。因此，实际特征选择并不是单纯追求离线指标最大化，而需要综合衡量效果收益与系统成本之间的权衡。

随着大模型和多模态技术逐渐进入推荐系统，特征成本的内涵也在不断扩展。例如，在探索 LLM4Rec 和多模态推荐时，可以通过 LLM 或 VLM 为用户和物品生成 Embedding、语义标签（Tag）等高阶特征。这类特征虽然能够引入传统 ID 特征难以表达的语义信息，但其生成通常需要较大的离线计算资源，并且 Embedding 维度较高时还会进一步增加存储和传输成本。类似地，如果从用户长期历史行为序列中提取兴趣特征，例如直接处理长度达到 2 万的行为序列，无论是在训练阶段构造特征，还是在线推理阶段实时计算，都会带来较高的计算和延迟开销。

此外，一些看似简单的特征也可能具有较高的隐性成本。例如，依赖实时流式数据计算的高频统计特征，需要持续消耗实时计算资源；跨多个特征服务聚合得到的特征，会增加 RPC 调用次数和服务依赖；高维 Embedding 或大量序列特征则会显著增加显存、内存以及网络带宽压力。因此，在引入高成本特征时，需要同时关注其离线增益、线上 A/B 收益以及新增系统成本，从而评估特征的实际 ROI。

最终，工业特征选择关注的并不是“是否能够带来收益”，而是“带来的收益是否值得其成本”。一个特征即使能够提升离线指标，如果需要显著增加训练资源、在线延迟或者系统复杂度，其实际价值也可能有限；反之，一些计算成本极低、但能够带来稳定收益的特征，即使单个特征的指标增益并不显著，也可能具有较高的工程价值。因此，特征工程需要从单纯追求模型效果，进一步转向效果、成本与 ROI 的综合优化。

15.6.4 特征质量

特征质量决定了一个特征是否能够长期、稳定地为推荐模型提供有效信息。一个在离线实验中看起来非常有效的特征，如果存在低覆盖率、高缺失率、取值异常、更新不及时或者上下游数据不一致等问题，在线上环境中往往很难稳定发挥作用。因此，工业推荐系统不仅需要关注特征是否有效，还需要持续关注特征是否稳定、可靠且符合预期。

对于一个特征 f，最基本的质量指标是覆盖率（Coverage），可以定义为：

$$\text{Coverage}(f) = \frac{N_{\text{valid}}(f)}{N_{\text{request}}} \tag{15.107}$$

其中，$N_{\text{valid}}(f)$ 表示能够正常获取有效取值的样本数量，N_{request} 表示总请求数量。除了覆盖率和缺失率之外，还需要关注特征基数（Cardinality）、取值分布、均值与方差、极值比例以及不同时间窗口下的分布变化。例如，一个原本取值范围稳定的统计特征突然出现大规模极端值，或者某个离散特征的基数在短时间内异常增长，都可能意味着上游数据源、特征产出 Pipeline 或者计算逻辑发生了异常。

对于时序特征而言，更新时效性（Freshness）同样非常重要。某些用户行为、实时统计或者热点相关特征需要持续更新，如果特征因为上游任务延迟、缓存异常等原因长时间没有更新，即使其取值本身没有明显异常，也可能已经无法准确反映用户和物品的当前状态。因此，工业特征质量通常需要同时关注“**有没有值、值是否正常、值是否及时**”三个维度。

更重要的是，特征质量并不是一次性的离线检查，而应该成为特征生命周期中的持续监控过程。在工业级推荐系统中，模型通常依赖数百甚至数千个特征，不同特征的上游数据源、计算逻辑和更新周期差异很大，有些特征通过离线任务产出，有些则通过实时流式计算或者在线服务实时获取。如果每个特征都由模型开发者独立维护，特征质量很容易成为整个推荐链路中的薄弱环节。

因此，成熟的推荐系统通常会建设统一的特征计算与管理平台，将特征产出、存储、服务以及质量监控进行统一管理，并由专门的基础架构或推荐架构团队维护。平台可以针对每个特征持续监控覆盖率、缺失率、基数、分布、更新延迟等指标，并通过阈值告警或者历史分布对比及时发现异常。进一步地，还可以建立特征血缘（Feature Lineage），记录特征依赖的数据源、计算任务和下游模型，从而在特征异常时快速定位影响范围。

这种平台化建设对于推荐系统的故障排查尤其重要。假设线上某个核心业务指标突然下降，进一步发现某个目标的 AUC 出现异常下降，如果问题最终是由某个重要特征的覆盖率下降、取值分布异常或者特征更新延迟导致的，那么在缺少统一特征质量监控的情况下，排查人员可能需要从大量特征中逐个检查，故障定位成本非常高。而如果已经建立了完善的特征质量监控和血缘体系，则可以沿着“**业务指标 → 模型指标 → 特征 → 上游数据 Pipeline**”快速定位异常，大幅缩短故障发现和恢复时间。

此外，特征质量还需要关注训练与线上服务的一致性。例如，训练数据中的特征由离线 Pipeline 计算，而线上特征由实时服务生成，如果二者存在计算逻辑、默认值、归一化方式或者更新时间上的差异，就可能产生离在线不一致问题，使模型离线效果正常但线上效果明显下降。因此，特征质量监控不应只检查单个特征的数值是否正常，还需要关注训练、评估和线上服务之间的数据一致性。

从工业实践来看，特征质量体系往往是在推荐系统不断演进的过程中逐步建立起来的。很多团队在模型出现线上异常、特征异常导致重大故障之后，才开始补充覆盖率监控、分布监控和特征血缘等能力。相比事后排查，更成熟的做法是将特征质量监控作为特征上线的必要环节，使每个特征从产生开始就具备可监控、可追踪、可告警和可回溯的能力。只有这样，特征工程才能从单纯追求模型效果，进一步成为支撑推荐系统长期稳定运行的基础设施。

15.6.5 数据漂移

推荐系统并不是运行在一个静态的数据环境中。用户兴趣、用户结构、物品供给、内容生态以及业务策略都在持续变化，使得训练数据与线上数据的分布可能随着时间发生明显差异。假设训练数据中的特征分布为 $P_{\text{train}}(X)$，线上数据分布为 $P_{\text{online}}(X)$，当

$$P_{\text{train}}(X) \neq P_{\text{online}}(X) \tag{15.108}$$

就可能产生数据漂移（Data Drift）。严重的数据漂移不仅会导致模型预测效果下降，还可能使原本重要的特征逐渐失去价值，甚至使模型训练时学习到的规律与线上真实环境产生明显偏差。

在工业推荐系统中，数据漂移通常可以从多个维度进行观察。首先是**用户侧漂移**。以短视频 APP 为例，随着产品不断发展，用户规模和用户结构可能发生长期变化，例如早期用户主要集中在一二线城市，而随着产品逐渐向三四线城市及更广泛人群渗透，下沉用户的占比不断提高，使得用户所在省份、城市等级、年龄、活跃程度等特征的分布持续发生变化。类似地，暑假、寒假以及节假日期间，学生或年轻用户的活跃程度可能明显增加，也会导致用户群体结构和兴趣分布发生短期漂移。

其次是**物品和内容侧漂移**。推荐系统长期运行过程中，内容供给本身也会发生变化。例如，如果系统长期偏向推荐长视频，可能逐渐导致线上曝光内容中长视频的占比提高，使得视频 Duration 的分布不断向更长的方向移动；某些内容垂类快速增长，也可能使不同 Category、Tag 以及内容风格的占比发生变化。随着内容生态不断演化，训练数据中原本常见的内容类型可能逐渐减少，而新的内容类型不断出现，从而形成明显的物品侧分布漂移。

第三类是**场景和业务侧漂移**。推荐系统的流量并不总是稳定的，节假日、大促、热点事件以及业务策略调整都可能导致线上数据分布在短时间内发生剧烈变化。例如，电商大促期间流量可能出现短时间激增，用户的访问目的、商品结构和交互行为都会与日常存在明显差异；节假日期间用户使用时长、访问时间以及消费内容也可能发生变化。这类漂移往往具有较强的周期性或者突发性，因此不能简单依赖长期训练数据进行覆盖。

除了特征层面的分布变化，还需要特别关注 **Label 层面的漂移（Label Drift）**。例如在短视频推荐中，某一时期用户整体观看时长提升，可能导致 CTR、长播率、有效观看率等 Label 的分布发生变化。在电商推荐中，大促期间用户购买意愿显著增强，商品 CTR、CVR 等指标可能明显高于日常水平。如果模型仍然使用长期历史数据学习到的 Label 分布，就可能无法准确反映当前业务场景下的用户行为。

更值得注意的是，一些 Label 的定义本身也可能随着内容生态变化而失效。例如，早期短视频内容整体较短时，可以使用固定的 5 秒、10 秒或者 30 秒阈值定义短播、长播等行为；但随着平台内容逐渐变长，整体 Duration 分布发生变化，原有阈值可能不再具有相同的业务含义。此时即使模型本身没有发生变化，Label 的正负样本比例以及 Label 与真实用户价值之间的关系也可能发生漂移。因此，数据漂移不仅是“数据变了”，还可能意味着过去有效的指标定义和建模假设已经不再适用。

针对上述问题，工业推荐系统通常需要同时监控特征分布漂移、用户和物品结构漂移、场景流量漂移以及 Label 分布漂移。在特征层面，可以通过 PSI、KL 散度等统计指标比较训练集与线上数据的分布；在 Label 层面，则可以持续监控 CTR、CVR、观看时长、长播率等核心 Label 的均值、分位数和分布变化。同时，还需要结合模型 AUC、Calibration 以及线上 A/B 实验结果判断分布变化是否已经实际影响模型效果。

因此，数据漂移也意味着特征选择并不是一次性的模型训练步骤，而是一个需要持续迭代的动态过程。一个特征在模型训练阶段可能具有较高的信息增益，但随着用户结构、内容生态和业务场景变化，其边际价值可能逐渐下降；反之，一些过去并不重要的新特征，也可能随着新的用户行为和内容形态出现而变得越来越重要。换言之，特征的重要性本身也会随着时间变化，工业推荐系统需要通过持续的数据监控、特征评估和模型迭代，使特征体系能够不断适应真实业务环境的变化。

15.6.6 特征泄露

特征泄露（Feature Leakage）是工业推荐系统中特别需要警惕的问题。它通常指特征中包含了在实际预测时刻不可获得的信息，导致离线训练和评估结果异常优秀，但上线后无法复现。例如，对于 t 时刻需要预测用户是否点击 Item，如果特征中错误地包含了 $t+1$ 时刻之后产生的行为统计，那么模型实际上已经“看到了未来”。这种特征虽然可能具有非常强的预测能力，但其提升并不是真实可用的模型能力。

最常见的一类泄露来自**未来行为泄露**。例如，在预测用户当前是否点击某个视频时，如果用户点击次数、最近活跃时间、兴趣标签等特征统计包含了当前曝光之后产生的行为，那么这些特征实际上已经使用了 Label 发生

之后的信息。类似地，如果统计“用户未来 7 天是否活跃”、“用户未来是否购买”等信息作为输入，也会直接将预测目标或其后验信息泄露给模型。

另一类常见问题是**统计窗口泄露**。例如在构造“用户过去 7 天点击某 Category 的次数”时，正确的统计窗口应该截止到预测时刻 t：

$$\text{Feature}(t) = \text{Agg}\left(a_\tau \mid t - 7d \le \tau < t\right) \tag{15.109}$$

如果特征计算任务直接使用了包含 t 时刻之后数据的完整窗口，例如离线任务每天重新统计过去 7 天的数据，就可能无意中将预测时刻之后发生的行为加入训练样本。对于推荐系统中大量存在的实时统计、滑动窗口和异步聚合特征，这类问题尤其容易发生。

在用户行为序列建模中，**Action List 的时间错位**是一个更加隐蔽的问题。假设训练样本对应用户在过去某个时刻 t 发起的一次推荐请求，模型需要利用当时已经发生的行为预测该请求中的 Item。但如果训练过程中不是按照样本发生时刻读取用户行为，而是实时查询用户当前最新的长度为 2 万的完整 Action List，那么得到的实际上是：

$$\text{ActionList}_{\text{current}} \supset \text{ActionList}_{\text{at } t} \tag{15.110}$$

其中包含了大量 t 时刻之后产生的用户行为。这样一来，虽然训练样本本身来自过去，但输入特征却来自“未来”，模型就能够间接看到用户在该请求之后的行为。

这种问题在训练数据和特征服务解耦、实时特征在线查询以及长行为序列建模场景中尤其容易出现。例如，一个训练样本记录的是用户在昨天观看某视频之前的状态，但训练任务执行时，用户已经产生了数百甚至数千条新的观看、点赞和关注行为。如果此时直接读取最新的 2 万条 Action List 计算用户兴趣 Embedding，那么这些最新行为就会进入昨天样本的特征中。离线训练时模型因此能够利用未来行为解释过去的 Label，最终可能获得异常优秀的 AUC；但线上预测时只能使用当前时刻之前已经发生的行为，两者的特征构造方式并不一致，从而产生严重的训练推理不一致（Training-Serving Skew）问题。

因此，对于每个特征，都需要明确其严格的时间边界：

$$\text{特征产出时间} \rightarrow \text{预测时间} \rightarrow \text{标签时间} \tag{15.111}$$

更准确地说，特征在预测时刻 t 的可用信息集合应该满足：

$$\mathcal{F}(t) \subseteq \mathcal{I}_{\le t} \tag{15.112}$$

即模型输入只能依赖预测时刻 t 之前已经产生并且在真实线上系统中能够获取的信息。对于行为序列，还需要保证训练阶段使用的序列截断位置与线上请求时刻一致，而不能简单使用当前最新的完整序列。

特征泄露还可能来自数据处理和 Pipeline 层面。例如，先使用完整数据集计算归一化参数、统计特征或者目标编码，再划分训练集和验证集；或者离线任务在 Label 生成完成之后才统一计算特征，却将这些特征回填到更早的训练样本中。这些操作在代码层面可能完全合理，但从时间因果关系来看都可能引入未来信息。

因此，特征泄露本质上并不是一个简单的特征选择问题，而是数据时间边界、信息可获得性以及因果关系管理问题。一个特征即使具有极高的预测能力，只要无法在真实预测时刻获得，就不应该被模型使用。工业推荐系统在构建特征 Pipeline 时，应尽可能采用 Point-in-Time Correctness 的方式生成训练特征，并对实时特征、行为序列和窗口统计建立明确的时间截断机制，从源头上保证“训练时看到的，就是线上当时能够看到的”。

15.6.7 特征治理

当推荐系统中的特征数量从几十、几百增长到数千甚至数万以后，仅依靠模型开发人员人工维护特征已经变得越来越困难。因此，工业推荐系统需要进一步建立特征治理（Feature Governance）体系，对特征进行统一管理。一个完整的特征治理体系通常需要记录特征的：

- **定义**：特征的业务含义和计算逻辑；
- **负责人**：特征的负责团队和维护人员；
- **来源**：特征的数据来源；

- **新鲜度**：特征的更新频率；
- **覆盖率**：特征在线覆盖情况；
- **成本**：特征的计算和服务成本；
- **质量**：特征的数据质量；
- **漂移**：特征的数据分布变化；
- **使用情况**：特征被哪些模型和业务使用；
- **生命周期**：特征的创建、迭代、废弃和下线状态。

因此，一个成熟的特征平台实际上不仅是特征存储（Feature Store），更应该承担特征发现（Feature Discovery）、特征监控（Feature Monitoring）、特征质量（Feature Quality）以及特征生命周期管理（Feature Lifecycle Management）等职责。通过统一治理，可以避免大量重复特征长期存在，也可以及时发现失效特征、异常特征以及高成本低收益特征。

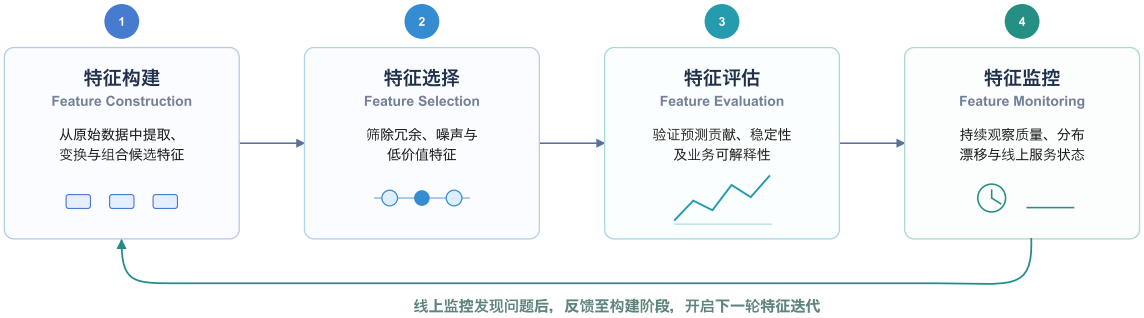


图 15.12: 特征生命周期管理示意图。

最终，特征工程也由早期的“特征构建（Feature Construction）”逐渐发展为一个完整的生命周期管理过程，如图所示。从整个特征工程的发展来看，特征选择也正在逐渐从传统的“删除无用特征”转变为更加系统的特征生命周期管理（Feature Lifecycle Management）。对于早期推荐系统而言，特征工程的主要目标是不断构造新的有效特征；而当特征规模达到一定程度后，如何判断特征价值、控制特征成本、监控特征质量并及时淘汰失效特征，反而成为更加重要的问题。与此同时，随着 Attention、Gating、Transformer 等模型结构的发展，部分特征选择能力也开始逐渐内化到模型本身，使得“特征是否重要”不再完全依赖人工规则，而可以由模型根据用户、物品和上下文动态决定。这种从人工特征工程向模型自动特征选择和表示学习的演进，也为后续更大规模的推荐模型发展奠定了基础。

15.7 本章小结

在本章节中，我们系统介绍了推荐系统中的特征工程。特征工程的核心并非简单地对原始数据进行清洗和编码，而是将用户、物品、场景与交互行为中的业务信息，转化为模型能够稳定利用的特征表示。本章首先从特征体系出发，介绍了用户侧特征、物品侧特征、上下文特征、行为特征以及多模态特征等信息来源，并说明了不同类型特征在刻画长期偏好、短期兴趣、物品内容与当前推荐场景中的作用。

随后，我们讨论了 Sparse、Dense、Sequence 和 Embedding 等特征表示形式，以及特征体系的工业化设计、Embedding 表征的作用和从传统特征工程到表征学习的演进。同一业务信息可以通过统计值、离散 ID、行为序列或 Dense 向量等多种形式进入模型，具体选择需要兼顾信息表达能力、数据规模、计算成本和线上可获取性。尤其是用户行为序列和多模态 Embedding，使模型能够从历史交互与非结构化内容中学习更加丰富的兴趣与语义表示。

在用户行为建模与实时特征部分，本章进一步介绍了多时间窗口行为统计、用户画像和实时上下文信息。长期统计特征有助于刻画相对稳定的兴趣与质量信号，短期行为序列和实时特征则能够及时捕捉兴趣迁移、热点变化与当前会话意图。要让这类特征在线上真正发挥价值，还需要处理好特征更新频率、计算延迟、缓存与服务

稳定性等工程问题，并保证训练样本与线上请求在特征计算逻辑和时间边界上的一致。

在特征交互建模部分，我们梳理了近年来工业级推荐系统中的重要方法及其发展脉络，依次介绍了 DCN、DCN-V2、AutoInt、FiBiNet、AFN、FED、CAN、FINT、GDCN 和 IPA 等模型。总体而言，特征交互建模正逐步从依赖人工设计的显式交叉，转向由模型自动学习不同特征之间的非线性关系、组合模式与动态贡献。Attention、Gating 等机制也使模型能够根据用户、物品和上下文，自适应地调整不同信息单元的重要程度，从而使特征工程与模型结构、表示学习之间的联系更加紧密。

最后，我们深入探讨了特征筛选建模的核心问题，包括特征重要性、特征冗余、特征成本、特征质量、数据漂移以及特征泄露等方面。工业推荐系统中的特征选择不能只追求离线指标提升，还需要综合评估增量收益、线上延迟、存储与计算资源、数据质量及长期维护成本。本章进一步提出了特征治理的概念：特征应从定义、生产、服务、监控到下线进行全生命周期管理，并通过特征血缘、质量监控、漂移检测、训练与线上一致性校验和持续迭代，确保特征体系能够适应不断变化的业务环境并持续创造价值。